SLOT 14

RETRY & CACHING, DATA PERSISTENCE & RESILIENCE

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Retry & Caching
- SharedPreferences Basics
- Local DB (SQLite/Hive/Drift)
- Stream

1 RETRY & CACHING

◈ A. Khái niệm cốt lõi sinh viên phải hiểu đúng

Retry là gì?

Là cơ chế **tự thử lại request API khi bị lỗi tạm thời**
Ví dụ: mạng chập chờn → gọi lại sau 2 giây
Không phải lỗi nào cũng retry (ví dụ 401 sai mật khẩu)

Caching là gì?

Là việc **lưu tạm dữ liệu đã lấy từ API để dùng lại**
Giúp:
Không gọi API liên tục
Mở app nhanh hơn
Xem được dữ liệu khi mất mạng

Memory cache vs Local cache

Memory: mất khi tắt app
Local storage: vẫn còn khi mở lại app

Resilience (khả năng chịu lỗi)

App không "vỡ" khi mất mạng
App vẫn usable trong điều kiện xấu

◈ B. Mục tiêu học tập cụ thể

Sau bài này sinh viên phải làm được:
Xử lý retry khi:
Timeout
Mạng chập chờn
Phân biệt:
Lỗi cần retry
Lỗi không nên retry
Cache dữ liệu danh sách sản phẩm
Hiển thị:
"Loading"
"Retry"
"Offline mode"
Tránh gọi API liên tục khi mở màn hình

◈ C. Ý nghĩa sư phạm

Nếu không có retry & caching:
App mất mạng → màn hình trắng
Người dùng phải reload liên tục
Server bị spam request
Trải nghiệm cực tệ
Liên hệ thực tế:
App shopping

PhucPT10@fpt.edu.vn

App tin tức
App danh sách sản phẩm
App todo có đồng bộ server

2 SHARED PREFERENCES (Key–Value Storage)

◈ A. Khái niệm cốt lõi sinh viên phải hiểu đúng

Persistent storage là gì?
Dữ liệu vẫn còn sau khi tắt app
Key–Value storage là gì?
Lưu dữ liệu dạng:
key: "token"
value: "abc123"
Giống như Map nhỏ trong ổ cứng
SharedPreferences dùng khi nào?
Lưu dữ liệu nhỏ:
token login
isFirstLaunch
theme mode
user settings
Không dùng để:
Lưu danh sách 500 sản phẩm
Lưu dữ liệu phức tạp

◈ B. Mục tiêu học tập cụ thể

Sau bài này sinh viên phải làm được:
Lưu token sau khi login
Đọc token khi mở app
Tự động skip màn login nếu đã đăng nhập
Lưu:
chế độ Dark mode
trạng thái đã onboarding
Ứng dụng thực tế:
App login
App ghi nhớ người dùng
App nhớ trạng thái toggle

◈ C. Ý nghĩa sư phạm

Nếu không có SharedPreferences:
Mỗi lần mở app phải login lại
Theme luôn reset
Không nhớ người dùng đã xem tutorial
→ App rất thiếu chuyên nghiệp.

3 LOCAL DATABASE (SQLite / Hive)

◈ Local Database là gì?

Là nơi lưu:
Danh sách dữ liệu lớn
Dữ liệu có cấu trúc
Có thể tìm kiếm, cập nhật, xóa
Hoạt động offline

PhucPT10@fpt.edu.vn

Không phụ thuộc vào network

📌 Ví dụ:

Danh sách sản phẩm
Đơn hàng
Task
Lịch sử giao dịch

👉 Dùng khi:

Dữ liệu là trung tâm của ứng dụng, không chỉ là cấu hình nhỏ lẻ.

◈ SQLite là gì?

Database quan hệ (Relational Database)
Có bảng (table), cột (column), khóa (primary key)
Truy vấn bằng SQL
Phù hợp dữ liệu có cấu trúc rõ

Flutter thường dùng package:

`sqflite`

📌 Phù hợp khi:

Cần truy vấn WHERE, JOIN
Có nhiều bảng liên quan
Cần filter & thống kê

◈ Hive là gì?

NoSQL local database
Không cần SQL
Lưu object nhanh
Dễ dùng hơn SQLite

Flutter package:

`Hive`

📌 Phù hợp khi:

Cần tốc độ nhanh
Dữ liệu không quá quan hệ phức tạp
Muốn code đơn giản
Lưu object trực tiếp

◈ Drift là gì?

Drift là:
SQLite nâng cấp với Type Safety + Code Generation + Reactive Query
Nó vẫn dùng SQLite bên dưới, nhưng:

Không phải viết SQL thủ công
Có sinh code tự động
Hạn chế lỗi runtime
Hỗ trợ reactive data bằng Stream

📌 Drift giúp:

Truy vấn phức tạp nhưng an toàn kiểu dữ liệu
Tự động cập nhật UI khi dữ liệu thay đổi
Tổ chức data layer chuyên nghiệp

👉 Phù hợp khi:

Dự án trung bình → lớn
Có nhiều business rule
Cần maintain lâu dài
Muốn kiến trúc production-ready

PhucPT10@fpt.edu.vn

◈ Stream là gì trong ngữ cảnh storage?

Stream là:
Luồng dữ liệu phát ra nhiều lần theo thời gian
Khác với Future:

Future	Stream
Trả về 1 lần	Trả về nhiều lần
Dùng cho thao tác 1 lần	Dùng cho dữ liệu thay đổi liên tục

Trong storage:
SQLite thuần → trả Future
Drift → có thể trả Stream
Ví dụ:
Khi thêm 1 item vào DB
Khi cập nhật trạng thái
Khi xóa dữ liệu
UI có thể tự cập nhật nếu dùng Stream.
📌 Stream là nền tảng của reactive UI.

◈ Local DB khác gì SharedPreferences?

SharedPreferences	SQLite / Hive / Drift
Key-value đơn giản	Lưu danh sách lớn
Dữ liệu nhỏ	Dữ liệu phức tạp
Không query phức tạp	Có tìm kiếm, filter
Không có quan hệ bảng	Có thể có quan hệ
Dùng cho settings	Dùng cho app data
Không reactive	Drift có reactive Stream

◈ SQLite vs Hive vs Drift – Hiểu đúng bản chất

Tiêu chí	SQLite (sqflite)	Hive	Drift
SQL	Có	Không	Có (an toàn kiểu)
Type safety	Thấp	Trung bình	Cao
Reactive	Không	Không mặc định	Có
Query phức tạp	Có	Hạn chế	Rất mạnh
Phù hợp	App vừa	App nhỏ-vừa	App production

◈ SO SÁNH TỔNG QUAN – ÁNH XẠ CÔNG NGHỆ

Mục đích	Flutter	Android (Java/Kotlin)	iOS (Swift)
Lưu key–value nhỏ	SharedPreferences	SharedPreferences	UserDefaults
SQLite thuần	sqflite	SQLiteOpenHelper	SQLite3
ORM trên SQLite	drift	Room	Core Data
NoSQL local	Hive	(ít dùng, có Realm)	Realm
Reactive DB	drift + Stream	Room + LiveData / Flow	Core Data + Combine
Quan sát dữ liệu thay đổi	Stream	LiveData / Flow	Combine / NotificationCenter
Chia sẻ dữ liệu giữa app	(hiếm dùng)	ContentProvider	App Groups / Core Data

PhucPT10@fpt.edu.vn

Điều sinh viên phải hiểu rõ

1. SharedPreferences không phải database.
2. SQLite là nền tảng quan hệ.
3. Hive đơn giản & nhanh nhưng không mạnh về query phức tạp.
4. Drift là lựa chọn chuyên nghiệp khi cần reactive & maintain lâu dài.
5. Stream là chìa khóa để UI luôn đồng bộ với dữ liệu.

Tư duy đúng cần hình thành

Dữ liệu không chỉ là lưu lại.
Dữ liệu phải được tổ chức, truy vấn, cập nhật và đồng bộ với UI một cách an toàn.

B. Mục tiêu học tập cụ thể

Sau bài này sinh viên phải làm được:

- Lưu danh sách todo vào local DB
- Lưu danh sách sản phẩm đã tải
- Đọc dữ liệu khi:
  - Không có mạng
- Cập nhật / xóa item
- Đồng bộ dữ liệu local ↔ server (mức cơ bản)

C. Ý nghĩa

Nếu không có local DB:

- Mất mạng → không xem được dữ liệu cũ
- Mỗi lần mở app phải tải lại toàn bộ
- Không thể build:
  - Offline todo app
  - App đọc báo offline
  - App shopping có cache sản phẩm

App thật 100% đều có local storage.

1 Drift (Reactive SQLite ORM trong Flutter)

Mục tiêu kiến thức

Sau phần này, sinh viên cần:

- Hiểu drift là gì và vì sao nó được dùng thay cho raw `sqflite`
- Biết cách:
  - Khai báo table bằng annotation
  - Sinh code tự động
  - Viết query cơ bản & nâng cao
  - Thực hiện JOIN, WHERE phức tạp
  - Sử dụng `watch()` để tạo dữ liệu reactive
- Hiểu được:

  drift = SQLite + Type Safety + Reactive Query + Code Generation

Drift giúp giải quyết vấn đề gì?

Trong app thực tế:

- Cần truy vấn phức tạp (lọc, sắp xếp, thống kê)
- Cần cập nhật UI tự động khi dữ liệu thay đổi
- Cần hạn chế lỗi runtime do sai tên cột
- Cần maintain code dài hạn

Drift giải quyết:

- Tránh viết raw SQL lặp lại
- Tránh lỗi sai kiểu dữ liệu

PhucPT10@fpt.edu.vn

Tự động rebuild UI khi DB thay đổi

2 Stream trong xử lý dữ liệu & storage

Mục tiêu kiến thức

Sinh viên cần hiểu:

Stream là gì? (luồng dữ liệu bất đồng bộ)

Sự khác nhau giữa:

`Future`

`Stream`

Stream được dùng trong:

Network

Database

Real-time UI

Vai trò Stream trong storage

Trong drift:

`watchTodos().watch()`

→ trả về `Stream<List<Todo>>`

UI dùng:

`StreamBuilder`

→ tự động rebuild khi DB thay đổi

Sau khi học xong phần này, sinh viên phải trả lời được:

Khi nào dùng Future? Khi nào dùng Stream?

Vì sao drift được xem là "production-ready database solution"?

Vì sao app lớn gần như luôn cần reactive data?

KẾT LUẬN TOÀN SLOT 14

Sau Slot 14 sinh viên phải hiểu:

App không chỉ là UI + API

App chuyên nghiệp cần:

Retry

Cache

Local storage

Offline strategy

reactive data

Sinh viên chuyển từ:

"App chạy được"

→

"App chịu lỗi tốt và dùng được trong môi trường thật"

TÀI LIỆU THAM KHẢO UY TÍN

Flutter Official

Flutter Official Documentation

https://docs.flutter.dev/

Packages

shared_preferences

https://pub.dev/packages/shared_preferences

sqflite

https://pub.dev/packages/sqflite

Hive

https://pub.dev/packages/hive

PhucPT10@fpt.edu.vn

Tài liệu dễ tiếp cận

Flutter Local Storage tutorials trên YouTube (search: "Flutter Hive tutorial for beginners")
Medium articles: "Flutter offline first strategy"
Các ví dụ chính thức trong pub.dev

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 RETRY & CACHING

◈ KHÁI NIỆM QUAN TRỌNG (PHẢI HIỂU ĐÚNG)

Retry ≠ gọi API lại vô hạn
- Retry là thử lại **có kiểm soát**
- Có:
  - Số lần giới hạn (ví dụ: 3 lần)
  - Thời gian chờ giữa các lần (delay)
- Không retry khi:
  - Sai mật khẩu (401)
  - Lỗi logic request

Retry dùng cho lỗi gì?
- Mạng chập chờn
- Timeout
- Server tạm thời lỗi (5xx)

Caching ≠ lưu mãi mãi
- Cache là lưu tạm để:
  - Tăng tốc
  - Giảm gọi API
  - Dùng khi offline
- Cache cần có chiến lược:
  - Khi nào refresh?
  - Khi nào xóa?

TTL (Time To Live)
- Cache chỉ hợp lệ trong 1 khoảng thời gian
- Ví dụ: cache danh sách sản phẩm 5 phút

◈ KỸ THUẬT PHẢI THÀNH THẠO

Sinh viên bắt buộc làm được:
- Viết logic retry đơn giản:
  - try/catch
  - delay giữa các lần retry
  - giới hạn số lần
- Phân biệt:
  - Lỗi cần retry
  - Lỗi hiển thị ngay cho user
- Cache dữ liệu danh sách sản phẩm vào local
- Hiển thị:
  - Loading
  - Retry button
  - Offline message
- Khi mở app:
  - Hiển thị cache trước
  - Sau đó gọi API refresh

👉 Trong project thật, sinh viên phải tự xây được:
- Product List có cache
- Todo app xem được khi mất mạng

◈ LỖI THƯỜNG GẶP

❌ Retry vô hạn → app treo

🔍 Nguyên nhân: Không giới hạn số lần retry

PhucPT10@fpt.edu.vn

❌ Retry cả lỗi 401
🔍 Nguyên nhân: Không phân loại lỗi
❌ Cache nhưng không bao giờ refresh
🔍 Nguyên nhân: Không hiểu cache chỉ là tạm thời
❌ Gọi API mỗi lần build()
🔍 Nguyên nhân: Chưa hiểu lifecycle widget

◈ BEST PRACTICES

Retry tối đa 2–3 lần
Có delay giữa các lần retry
Cache phải có TTL
Hiển thị cache trước → gọi API sau
Tách:
    Data layer
    UI layer

◈ CỐT LÕI vs NÂNG CAO

🟢 CỐT LÕI:
Retry có giới hạn
Cache danh sách đơn giản
Biết khi nào cần cache

🟡 NÂNG CAO:
Exponential backoff
Offline-first architecture
Repository pattern

◈ Ý NGHĨA SƯ PHẠM

Nếu không hiểu retry & caching:
App mất mạng → trắng màn hình
User bấm reload liên tục
Server bị spam
Trải nghiệm rất tệ

2 SHARED PREFERENCES (Key–Value Storage)

Package phổ biến:
shared_preferences

◈ KHÁI NIỆM QUAN TRỌNG (PHẢI HIỂU ĐÚNG)

SharedPreferences ≠ Database
    Chỉ lưu key–value đơn giản
    Không query phức tạp
    Không lưu object lớn
Dùng khi:
    Lưu token login
    Lưu isLoggedIn
    Lưu theme
    Lưu flag onboarding
Không dùng khi:
    Lưu danh sách todo
    Lưu 500 sản phẩm

PhucPT10@fpt.edu.vn

◈ KỸ THUẬT PHẢI THÀNH THẠO

Sinh viên phải làm được:

- Lưu token sau login
- Đọc token khi app khởi động
- Tự động skip màn login
- Lưu boolean (dark mode)
- Xóa token khi logout
- Xử lý async khi đọc SharedPreferences

👉 Ứng dụng trong project:

- App login có remember user
- App ghi nhớ theme

◈ LỖI THƯỜNG GẶP

❌ Lưu object phức tạp vào SharedPreferences
🔍 Nguyên nhân: Không hiểu nó chỉ là key–value đơn giản
❌ Quên await khi đọc dữ liệu
🔍 Nguyên nhân: Chưa nắm async
❌ Đọc SharedPreferences trong build()
🔍 Nguyên nhân: Không hiểu lifecycle

◈ BEST PRACTICES

- Chỉ lưu dữ liệu nhỏ
- Không lưu danh sách lớn
- Đọc dữ liệu ở initState hoặc trước khi runApp
- Tách logic lưu trữ khỏi UI

◈ CỐT LÕI vs NÂNG CAO

🟢 CỐT LÕI:

- Lưu token
- Lưu boolean
- Đọc khi mở app

🟡 NÂNG CAO:

- Secure storage (mã hóa)
- Dependency injection cho storage layer

◈ Ý NGHĨA SƯ PHẠM

Nếu không có SharedPreferences:

- Mỗi lần mở app phải login lại
- Không nhớ theme
- App rất thiếu chuyên nghiệp

3 LOCAL DATABASE (SQLite / Hive / Drift)

Package phổ biến:

```
sqflite
Hive
drift
```

◈ KHÁI NIỆM QUAN TRỌNG (PHẢI HIỂU ĐÚNG)

Local Database ≠ chỉ dùng cho dữ liệu lớn

Dùng khi dữ liệu:

- Có cấu trúc

PhucPT10@fpt.edu.vn

Có danh sách
Cần thêm / sửa / xóa
Cần tồn tại lâu dài
Cần hoạt động offline

SQLite (sqflite)

Có bảng
Có SQL
Phù hợp dữ liệu quan hệ
Truy vấn bằng raw SQL

👉 Phù hợp khi:
Dự án vừa
Query không quá phức tạp
Chấp nhận viết SQL thủ công

Hive

NoSQL
Không cần SQL
Lưu object nhanh
Dễ học hơn SQLite

👉 Phù hợp khi:
App nhỏ → trung bình
Không cần JOIN phức tạp
Ưu tiên tốc độ và đơn giản

Drift (SQLite nâng cao + Reactive)

Drift là:
SQLite + Type Safety + Code Generation + Reactive Query
Khác với sqflite:

sqflite	Drift
Viết SQL thủ công	Sinh code an toàn kiểu
Trả Future	Có thể trả Stream
Không reactive	Reactive built-in
Dễ sai tên cột	Compile-time safety

👉 Drift phù hợp khi:
Dự án trung bình → lớn
Có nhiều business rule
Có truy vấn phức tạp (filter nâng cao, join, thống kê)
Cần UI tự động cập nhật khi DB thay đổi
Muốn kiến trúc production-ready

SharedPreferences vs Local DB

SharedPreferences	SQLite / Hive / Drift
Token, setting	Danh sách dữ liệu
Key-value	Có cấu trúc
Không query phức tạp	Có tìm kiếm, filter
Không phù hợp dữ liệu lớn	Phù hợp dữ liệu lớn

PhucPT10@fpt.edu.vn

◈ KỸ THUẬT PHẢI THÀNH THẠO

Sinh viên phải làm được:

Với SQLite / Hive:

Tạo table / box
Insert / Update / Delete
Đọc danh sách khi mở app
Hiển thị dữ liệu offline
Không block UI khi truy vấn DB

Với Drift:

Khai báo table bằng annotation
Sinh code bằng build_runner
Viết query có điều kiện
Sử dụng `watch()` để trả về Stream
Hiển thị UI bằng `StreamBuilder`
Viết query có:
- WHERE
- ORDER BY
- JOIN
- COUNT / SUM

👉 Trong project thật:
Todo app offline
Product cache local
Dashboard cập nhật realtime khi thêm dữ liệu
Query danh sách "chưa hoàn thành + quá hạn"

◈ LỖI THƯỜNG GẶP

❌ Block UI khi query DB
🔍 Nguyên nhân: Không hiểu async

❌ Không await khi insert
🔍 Nguyên nhân: Tư duy đồng bộ

❌ Không xử lý khi DB chưa mở
🔍 Nguyên nhân: Không hiểu lifecycle app

❌ Lưu tất cả vào SharedPreferences
🔍 Nguyên nhân: Không phân biệt loại storage

❌ Với Drift: Quên regenerate code
🔍 Nguyên nhân: Không chạy build_runner sau khi đổi schema

❌ Không dùng Stream mà tự reload thủ công
🔍 Nguyên nhân: Chưa hiểu reactive data

PhucPT10@fpt.edu.vn

❌ Listen Stream trong build()
🔍 Nguyên nhân: Không hiểu lifecycle widget

◈ BEST PRACTICES

Với SQLite / Hive

Dùng DB cho danh sách
Dùng async/await đúng cách
Không thao tác DB trong build()
Tách Repository layer khỏi UI

Với Drift

Không truy cập DB trực tiếp từ Widget
Sử dụng Repository pattern
Dùng `watch()` cho dữ liệu hiển thị
Tách:
    Data source
    Repository
    UI
Có version & migration khi thay đổi schema
Không lạm dụng Stream cho dữ liệu chỉ fetch 1 lần

◈ CỐT LÕI vs NÂNG CAO

◍ CỐT LÕI:

Lưu danh sách todo
CRUD cơ bản
Đọc dữ liệu offline
Hiểu Future vs Stream

◍ NÂNG CAO:

Drift reactive query
Migration DB
Sync 2 chiều local ↔ server
Conflict resolution
Offline-first architecture

4 STREAM (Reactive Data Flow)

◈ KHÁI NIỆM QUAN TRỌNG (PHẢI HIỂU ĐÚNG)

Stream là:
Dữ liệu phát ra nhiều lần theo thời gian
Future:
    Trả 1 lần
Stream:
    Trả nhiều lần
Trong storage:
    SQLite thuần → Future
    Drift → Future + Stream

◈ Stream dùng khi nào?

Danh sách thay đổi liên tục

PhucPT10@fpt.edu.vn

Dashboard cần cập nhật tự động
DB thay đổi → UI phải thay đổi

Ví dụ:

Thêm task → list cập nhật ngay
Sửa trạng thái → dashboard cập nhật
Xóa item → UI tự refresh

◇ KỸ THUẬT PHẢI THÀNH THẠO

Sinh viên phải làm được:

Phân biệt FutureBuilder vs StreamBuilder
Dùng StreamBuilder đúng cách
Không listen Stream trong build()
Hủy Stream khi dispose()

◇ LỖI THƯỜNG GẶP

✕ Dùng Future thay vì Stream cho dữ liệu thay đổi liên tục

🔍 Nguyên nhân: Chưa hiểu reactive

✕ Gọi setState sau khi Stream update nhưng widget đã dispose

🔍 Nguyên nhân: Không hiểu lifecycle

✕ Stream bị leak

🔍 Nguyên nhân: Không dispose controller

◇ BEST PRACTICES

Chỉ dùng Stream cho dữ liệu thay đổi nhiều lần
Không tạo Stream mới mỗi lần build()
Kết hợp Drift + StreamBuilder cho UI reactive
Không để business logic trong Widget

◇ CỐT LÕI vs NÂNG CAO

◍ CỐT LÕI:

Hiểu Stream là reactive data
Dùng StreamBuilder đúng cách

◍ NÂNG CAO:

Combine nhiều Stream
Debounce / throttle
Reactive architecture (BLoC / Riverpod)

◍ TỔNG KẾT TƯ DUY BẮT BUỘC

Sau Slot 14 sinh viên phải có tư duy:

App thật luôn có:

Mạng chập chờn
Người dùng thoát app
Dữ liệu cần giữ lại
DB thay đổi liên tục

Không thể chỉ:

Gọi API → hiển thị → xong

Phải trả lời được:

Khi nào cần retry?

PhucPT10@fpt.edu.vn

Khi nào cần cache?
Khi nào dùng SharedPreferences?
Khi nào dùng SQLite?
Khi nào dùng Hive?
Khi nào nên dùng Drift?
Khi nào cần Stream thay vì Future?

SAU KHI HỌC XONG SLOT 14

Sinh viên phải làm được:

App login nhớ người dùng
App todo hoạt động offline
App sản phẩm có cache + TTL
App dùng Drift tự cập nhật UI
App không crash khi mất mạng
UI không bị block khi query DB

TƯ DUY QUAN TRỌNG NHẤT

App production không sợ mất mạng.
App production không mất dữ liệu.
App production không reload thủ công.
Dữ liệu phải được quản lý có chiến lược.

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

1 ỨNG DỤNG TRONG PHÁT TRIỂN DỰ ÁN FLUTTER THỰC TẾ

1. Retry được dùng khi nào?

Dùng khi:

API timeout
Mạng chập chờn
Server lỗi tạm thời (5xx)

Không dùng khi:

Sai mật khẩu (401)
Request sai format
Lỗi logic

Cách triển khai thực tế

Retry tối đa 2–3 lần
Có delay giữa các lần (1–2 giây)
Nếu vẫn lỗi → hiển thị nút "Thử lại"

Mục tiêu

App không "vỡ" khi mạng yếu
User không phải tự bấm reload 10 lần

2. Caching giúp gì cho app?

Trong thực tế:

Mở app → gọi API mất 3–5 giây
Mạng yếu → loading rất lâu
Mất mạng → trắng màn hình

Nếu có cache:

Hiển thị dữ liệu cũ ngay lập tức
Gọi API refresh ở background
Nếu offline → vẫn dùng được

Cache thường lưu ở:

Memory (tạm thời)
Local DB (bền vững hơn)

Mục tiêu:

Mở app nhanh
Giảm phụ thuộc network
Tăng UX

3. SharedPreferences dùng trong trường hợp nào?

Package phổ biến:

shared_preferences

Dùng để:

Lưu token đăng nhập
Lưu theme mode
Lưu isFirstLaunch
Lưu setting nhỏ

Không dùng để:

Lưu danh sách 1000 sản phẩm
Lưu object phức tạp

Trong project thật:

Login xong → lưu token
Mở app → kiểm tra token

PhucPT10@fpt.edu.vn

Logout → xóa token

◈ 4. SQLite / Hive dùng khi nào?

Package phổ biến:

sqflite
Hive

SQLite phù hợp khi:

Dữ liệu có quan hệ
Cần query phức tạp
Có filter / search

Hive phù hợp khi:

Lưu object nhanh
Dễ dùng
MVP / startup cần triển khai nhanh

Dùng khi:

Lưu danh sách todo
Cache danh sách sản phẩm
App cần offline hoàn toàn

◍ Nhấn mạnh quan trọng

Slot 14 là bước chuyển:
App demo → App có thể đưa ra thị trường
Nếu không có persistence:

App reload → mất dữ liệu
Mất mạng → unusable
User uninstall

◈ 5. Drift dùng khi nào?

Package:
`drift`

☑ Dùng khi:

Dự án có business logic phức tạp
Cần query nâng cao (filter nhiều điều kiện, JOIN, thống kê)
Cần UI tự động cập nhật khi DB thay đổi
Dự án trung bình → lớn
Cần maintain lâu dài

✕ Không dùng khi:

Chỉ lưu vài key nhỏ (dùng SharedPreferences)
App rất đơn giản (Hive đủ dùng)
Không cần query nâng cao

👉 Cách triển khai thực tế

Ví dụ trong app quản lý đơn hàng:

Table: orders
Table: customers
Table: order_items

Yêu cầu:

Lọc đơn chưa hoàn thành

PhucPT10@fpt.edu.vn

Tổng tiền > 500k
Sắp xếp theo ngày giảm dần
Dashboard tự cập nhật khi có đơn mới

Với Drift:
Viết query type-safe
Dùng `watch()` để trả về Stream
UI dùng StreamBuilder

🎯 Mục tiêu

Không viết SQL string thủ công
Tránh lỗi runtime do sai tên cột
UI tự cập nhật khi dữ liệu thay đổi
Kiến trúc rõ ràng (Data Layer tách biệt)

◈ 6. Stream giúp gì cho app?

✅ Dùng khi:

Danh sách thay đổi liên tục
Dashboard cần cập nhật realtime
Dữ liệu có thể bị thay đổi từ nhiều nguồn
Dùng Drift (reactive query)

❌ Không dùng khi:

Chỉ fetch dữ liệu 1 lần
Dữ liệu không thay đổi sau khi load

👉 Cách triển khai thực tế

Trong app Todo:
Thêm task
Sửa trạng thái
Xóa task

Nếu dùng Future:
Phải tự gọi lại fetch()
Dễ quên reload

Nếu dùng Stream:
DB thay đổi → Stream phát dữ liệu mới
UI tự rebuild

Ví dụ thực tế:
Dashboard hiển thị:
Tổng số task
Task hoàn thành
Task quá hạn

Khi user thêm 1 task:
→ Dashboard tự cập nhật, không cần setState thủ công.

🎯 Mục tiêu

UI luôn đồng bộ dữ liệu
Không reload thủ công
Kiến trúc reactive đúng chuẩn production

2 CÁC TÌNH HUỐNG THỰC TẾ

PhucPT10@fpt.edu.vn

📌 TÌNH HUỐNG 1 – App Bán Hàng

◈ Bối cảnh

App hiển thị danh sách sản phẩm từ API.

❗ Vấn đề

Mạng yếu → loading lâu
Mất mạng → trắng màn hình
Mỗi lần mở app → gọi API lại toàn bộ

🛠 Áp dụng Slot 14

1. Retry

Nếu timeout → retry 2 lần
Nếu vẫn lỗi → hiển thị cache

2. Caching

Lưu danh sách sản phẩm vào Hive
Khi mở app:
    Đọc local trước
    Sau đó gọi API refresh

3. Async xử lý đúng cách

await khi đọc DB
Không query trong build()

🎯 Kết quả

Mở app nhanh hơn
Offline vẫn xem được sản phẩm
Không spam server

📌 TÌNH HUỐNG 2 – App Todo Offline

◈ Bối cảnh

App quản lý công việc cá nhân.

❗ Vấn đề

Thêm task → nếu mất mạng thì mất luôn
Mở lại app → danh sách rỗng

🛠 Áp dụng Slot 14

1. Local Database

Lưu todo vào SQLite hoặc Hive
Insert / Update / Delete local trước

2. Đồng bộ server (cơ bản)

Khi có mạng → sync lại

3. Async đúng cách

await khi insert
Không block UI

🎯 Kết quả

App hoạt động hoàn toàn offline
User tin tưởng hơn
Không mất dữ liệu

PhucPT10@fpt.edu.vn

📌 TÌNH HUỐNG 3 – App Login

◈ Bối cảnh
App yêu cầu login trước khi sử dụng.

❗ Vấn đề
Mỗi lần mở app phải login lại
Không nhớ trạng thái người dùng

🛠 Áp dụng Slot 14
SharedPreferences
Sau login:
Lưu token
Khi mở app:
Kiểm tra token
Nếu tồn tại → skip login
Logout:
Xóa token

🎯 Kết quả
UX mượt hơn
App chuyên nghiệp hơn

📌 TÌNH HUỐNG 4 – App Tin Tức

◈ Bối cảnh
App đọc tin tức từ API.

❗ Vấn đề
API chậm
Mất mạng không đọc được bài đã xem

🛠 Áp dụng
Cache bài viết vào local DB
Retry nếu timeout
TTL: 10 phút refresh

🎯 Kết quả
Mở app gần như tức thì
Offline vẫn đọc được bài cũ

📌 TÌNH HUỐNG 5 – App Bán Hàng (Mở rộng nâng cao)

◈ Bối cảnh nâng cao
App có:
Sản phẩm
Danh mục
Giỏ hàng
Đơn hàng
Cần:
Filter theo danh mục

PhucPT10@fpt.edu.vn

Tìm kiếm theo từ khóa
Sắp xếp theo giá
Dashboard thống kê doanh thu

🛠 Áp dụng Drift + Stream

Drift
Tạo nhiều bảng
JOIN giữa sản phẩm và danh mục
Query tổng doanh thu theo tháng
Stream
watch danh sách sản phẩm
watch dashboard thống kê

Khi:

Có đơn hàng mới
Cập nhật trạng thái đơn

→ Dashboard tự cập nhật

🎯 Kết quả

Không cần reload toàn bộ app
UI luôn đúng dữ liệu
Code dễ maintain

📌 TÌNH HUỐNG 6 – App Todo Offline (Reactive Version)

◈ Vấn đề nâng cao

User:

Thêm task
Chỉnh sửa task
Lọc theo trạng thái
Xem thống kê

🛠 Áp dụng Drift + Stream

Drift lưu todo
watchTodos() trả về Stream
StreamBuilder hiển thị list

Khi:

Insert
Update
Delete

→ List tự cập nhật

🎯 Kết quả

Không gọi lại fetch()
Không dùng setState cho reload
UI mượt & chính xác

📌 TÌNH HUỐNG 7 – Dashboard Quản Lý Nội Bộ

◈ Bối cảnh

App quản lý công việc nội bộ công ty.
Dashboard hiển thị:

Tổng task

PhucPT10@fpt.edu.vn

Task quá hạn
Task hoàn thành hôm nay

❗ Vấn đề nếu không dùng Stream

Phải reload thủ công
Dữ liệu sai lệch khi nhiều user thao tác

🛠 Áp dụng

Drift query:
COUNT task
WHERE due_date < today
GROUP BY trạng thái
Trả về Stream
UI dùng StreamBuilder

🎯 Kết quả

Dashboard realtime
Không cần bấm refresh
Không sai lệch dữ liệu

3 CASE STUDY – MVP Startup Nội Bộ

🔑 Bối cảnh

Startup xây dựng app quản lý công việc nội bộ.
Ban đầu:
Chỉ gọi API
Không lưu local
Không retry

❗ Vấn đề

Mạng văn phòng không ổn định
User than phiền:
App hay loading
Mất task khi mất mạng
Server bị spam request

🛠 Áp dụng cải tiến

1. Retry có giới hạn
2 lần retry khi timeout
2. Cache danh sách task bằng Hive
3. Lưu local DB trước → sync sau
4. Tách Data Layer khỏi UI

✅ Kết quả

App mở nhanh hơn 60–70%
Không crash khi mất mạng
User hài lòng hơn
Server giảm tải đáng kể

4 CASE STUDY – MVP Startup Nội Bộ (Nâng cấp Production)

PhucPT10@fpt.edu.vn

🔑 Vấn đề nâng cao

Sau khi đã có:

- Retry
- Cache
- Hive

App vẫn gặp:

- Logic query rối
- Dashboard không realtime
- Code SQL khó maintain

🛠 Cải tiến thêm Drift + Stream

- Chuyển từ Hive sang Drift cho phần task phức tạp
- Tách Repository layer
- Dùng watch() cho dashboard

✅ Kết quả

- Code rõ ràng hơn
- Query phức tạp nhưng an toàn
- UI reactive
- Dễ mở rộng thêm tính năng

4 BÀI HỌC RÚT RA (LESSONS LEARNED)

◈ Điều quan trọng nhất

Data không chỉ nằm ở API, Lưu được dữ liệu

Luôn nghĩ đến:

- Mất mạng
- Mở lại app
- Network chậm
- Lưu có cấu trúc
- Truy vấn hiệu quả
- Đồng bộ UI tự động

◈ Sai lầm phổ biến

❌ Retry vô hạn
→ App treo

❌ Cache nhưng không refresh
→ Hiển thị dữ liệu cũ mãi

❌ Lưu list lớn vào SharedPreferences
→ App chậm, lỗi

❌ Không await khi đọc DB
→ UI hiển thị sai

❌ Query DB trong build()
→ Block UI

❌ Dùng Hive cho business logic phức tạp
→ Query khó mở rộng

❌ Không dùng Stream cho dashboard
→ UI sai lệch dữ liệu

❌ Query Drift trong UI layer
→ Kiến trúc rối

PhucPT10@fpt.edu.vn

❌ Tạo Stream trong build()
→ Memory leak

◈ Nguyên tắc lâu dài

- Tách UI và Data Layer
- Retry có kiểm soát
- Cache có chiến lược
- Không phụ thuộc 100% vào API
- Thiết kế theo hướng resilient
- Drift cho app có logic phức tạp
- Stream cho dữ liệu thay đổi nhiều lần
- Repository pattern bắt buộc khi app lớn
- Không để UI biết chi tiết DB

🎯 TƯ DUY PHẢI HÌNH THÀNH SAU SLOT 14

Sinh viên phải tự hỏi khi xây app:

Nếu mất mạng thì sao?
Nếu user thoát app thì dữ liệu còn không?
Nếu API lỗi thì xử lý thế nào?
Có đang spam server không?
Nếu dữ liệu thay đổi liên tục thì sao?
Nếu nhiều bảng liên quan thì dùng gì?
Có cần UI tự cập nhật không?
Có đang reload thủ công không?
Kiến trúc đã tách Data Layer chưa?

🚀 KẾT LUẬN

App tốt không phải là app:
"Chạy được khi mọi thứ hoàn hảo"

- Có retry
- Có cache
- Có local DB

Mà là app:
"Vẫn hoạt động khi mạng không hoàn hảo"

- Có kiến trúc data rõ ràng
- Có reactive data flow
- Có query an toàn
- Không phụ thuộc vào việc reload thủ công

Sau phần này, sinh viên phải:

Biết khi nào dùng SharedPreferences
Biết khi nào dùng SQLite / Hive/Drift
Biết reactive data flow
Biết viết retry an toàn
Biết cache đúng cách
Có tư duy xây app thật

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT
PHẦN A. RETRY & RESILIENCE

I. BẢN CHẤT CỦA LỖI MẠNG TRONG MOBILE APP

◈ Mạng không ổn định là trạng thái bình thường

Trong mobile:

Người dùng chuyển từ WiFi → 4G
Đi thang máy → mất sóng
Server bảo trì
API quá tải

👉 **Network failure không phải ngoại lệ – mà là điều tất yếu.**

◈ Vì sao mobile app không thể tin tưởng network 100%?

Vì:

Network phụ thuộc môi trường vật lý
API phụ thuộc server
User không kiểm soát được kết nối

Khác với web app:

Web reload là bình thường
Mobile reload gây trải nghiệm tệ

◈ Liên hệ thực tế

App ngân hàng: không thể crash khi mạng chập chờn
App thương mại điện tử: không thể mất giỏ hàng
App đặt xe: không thể đứng im khi API timeout

👉 Mobile app phải **chịu lỗi tốt hơn web**.

II. RETRY LOGIC – NGUYÊN LÝ HỌC THUẬT

◈ Retry là gì?

Retry là:
Thử lại request khi lỗi tạm thời xảy ra

◈ Retry có điều kiện

Retry chỉ nên xảy ra khi:

Timeout
Server 5xx
SocketException

Không retry khi:

401 Unauthorized
400 Bad Request

◈ Retry có giới hạn (Max Attempt)

Ví dụ:

Tối đa 3 lần
Mỗi lần cách nhau 2 giây

● Vì sao retry vô hạn gây nguy hiểm?

App treo
Pin hao
Spam server
Loop vô tận

PhucPT10@fpt.edu.vn

Nguyên nhân sinh viên mắc lỗi này:
Không phân biệt lỗi tạm thời và lỗi logic
Không đặt giới hạn số lần

◈ Exponential Backoff là gì? (giải thích đơn giản)

Thay vì:
Retry mỗi 1 giây
Thì:
Lần 1: 1 giây
Lần 2: 2 giây
Lần 3: 4 giây
Mục đích:
Giảm áp lực server
Tăng khả năng phục hồi

III. RESILIENCE DESIGN

◈ Resilient App là gì?

Là app:
Không crash
Không đứng im
Có fallback data
Không phụ thuộc hoàn toàn network

◈ Nguyên tắc thiết kế

Không crash khi API lỗi
Không block UI
Có dữ liệu dự phòng (cache)
Cho user biết trạng thái

◈ UX quan trọng hơn "đúng tuyệt đối"

Ví dụ:
Hiển thị dữ liệu cũ 5 phút trước
→ tốt hơn màn hình trắng
👉 Trong mobile, trải nghiệm mượt quan trọng hơn tính "real-time tuyệt đối".

PHẦN B. CACHING – TƯ DUY LƯU TẠM THỜI

I. CACHE LÀ GÌ? (BẢN CHẤT)

Cache là:
Lưu tạm dữ liệu để dùng lại nhanh hơn

◈ Cache ≠ lưu vĩnh viễn

Cache có thể hết hạn
Cache có thể bị xóa

◈ Cache ở đâu?

RAM (in-memory)
Local storage (DB, file)

◈ TTL (Time To Live)

TTL là:

PhucPT10@fpt.edu.vn

Thời gian dữ liệu được coi là hợp lệ
Ví dụ:
Cache danh sách sản phẩm 5 phút
Cache bài viết 10 phút

II. CHIẾN LƯỢC CACHE TRONG APP THẬT

◈ 1. Cache-first

Đọc cache trước
Nếu không có → gọi API
Phù hợp:
App tin tức

◈ 2. Network-first

Gọi API trước
Nếu lỗi → dùng cache
Phù hợp:
App ngân hàng

◈ 3. Offline-first (giới thiệu)

Dữ liệu chính nằm ở local
Network chỉ để sync
Phù hợp:
App Todo
App quản lý công việc

III. LỖI TƯ DUY PHỔ BIẾN

❌ Cache nhưng không invalidate
→ Hiển thị dữ liệu cũ mãi
❌ Cache dữ liệu nhạy cảm
→ Rủi ro bảo mật
❌ Nhầm cache với database
→ Thiết kế sai tầng dữ liệu

PHẦN C. LOCAL PERSISTENCE TRONG FLUTTER

I. PERSISTENT STORAGE LÀ GÌ?

◈ So sánh

In-memory	Persistent
Mất khi app đóng	Không mất
setState	DB / file
Dùng tạm thời	Dùng lâu dài

◈ Vì sao mobile app luôn cần persistent storage?

Vì:
User đóng app bất cứ lúc nào
OS có thể kill app
App cần nhớ trạng thái

PhucPT10@fpt.edu.vn

II. SHAREDPREFERENCES – KEY-VALUE STORAGE

Package:

shared_preferences

◇ Bản chất

Lưu key-value
Dữ liệu nhỏ
Không phải database

◇ Dùng khi

Lưu token
Lưu setting
Lưu flag

● Vì sao không lưu list lớn?

Không có query
Không tối ưu cho object phức tạp
Dễ gây chậm

● Sai lầm phổ biến

Lưu JSON dài hàng trăm dòng
Quên await
Đọc trong build()

III. SQLITE – RELATIONAL DATABASE

Package:

sqflite

◇ Cơ chế

Table
Row
Column
Query bằng SQL

◇ Ưu điểm

Query mạnh
Phù hợp dữ liệu có quan hệ

◇ Nhược điểm

Phải mapping object
Phức tạp hơn Hive

◇ Khi nào cần relational DB?

Dữ liệu có liên kết:

User – Order
Product – Category

IV. HIVE – NOSQL LOCAL DATABASE

Package:

Hive

PhucPT10@fpt.edu.vn

◈ Khái niệm

Box = vùng lưu trữ
Lưu object Dart trực tiếp
Không cần SQL

◈ So sánh Hive vs SQLite

Tiêu chí	Hive	SQLite
Dễ học	Cao	Trung bình
Query phức tạp	Hạn chế	Mạnh
Mapping object	Dễ	Cần viết thêm
Phù hợp MVP	Rất tốt	Tốt

V. ASYNC & CONCURRENCY TRONG LOCAL STORAGE

◈ Vì sao đọc DB là async?

Vì:
Truy cập disk chậm hơn RAM
Không được block main thread

◈ Nếu không await?

UI hiển thị dữ liệu rỗng
Lỗi logic ngầm

◈ Block UI là gì?

Là khi:
Main thread bị giữ
Animation đứng
App lag

◈ Event Loop & Future (giải thích đơn giản)

Flutter dùng:
Event loop
Future
async/await
Mục tiêu:
Không làm chậm UI
Xử lý song song I/O

PHẦN D. THIẾT KẾ DATA LAYER CƠ BẢN

I. TÁCH DATA LAYER KHỎI UI

◉ UI không nên:

Truy cập DB trực tiếp
Retry trong widget
Gọi API trong build()

◈ Separation of Concerns

Mỗi tầng làm một việc:
UI → hiển thị
Repository → xử lý logic

PhucPT10@fpt.edu.vn

Data source → DB / API

II. LUỒNG DỮ LIỆU TRONG APP THẬT

```
UI
 ↓
Repository
 ↓
Local DB / API
```

◈ Vì sao cần tầng trung gian?

Dễ test
Dễ thay đổi DB
Dễ thêm cache

◍ Nếu không tách?

Code rối
Khó bảo trì
Không mở rộng được

PHẦN E. LỖI THƯỜNG GẶP & PHÂN TÍCH HỌC THUẬT

✕ Retry loop vô hạn

Nguyên nhân:
Không có max attempt
Ảnh hưởng:
Hao pin
Spam server
Thiết kế đúng:
Retry có giới hạn
Có delay

✕ Cache dữ liệu nhạy cảm

Nguyên nhân:
Không phân loại data
Ảnh hưởng:
Rủi ro bảo mật
Thiết kế đúng:
Token nên mã hóa
Không cache thông tin nhạy cảm

✕ Block main thread

Nguyên nhân:
Query DB đồng bộ
Xử lý nặng trong UI
Ảnh hưởng:
App lag
Thiết kế đúng:
async/await
Tách logic ra khỏi build()

PhucPT10@fpt.edu.vn

PHẦN 1 – RETRY LOGIC (Có Giới Hạn)

Mục tiêu kỹ thuật

Sau phần này sinh viên phải:

- Viết được retry tối đa 2–3 lần
- Biết phân biệt lỗi retry được và không
- Không tạo loop vô hạn

Bước 1 – Hiểu bản chất retry

Retry dùng khi:

- Timeout
- SocketException
- Lỗi 5xx

Không retry khi:

- 401 Unauthorized
- 400 Bad request

Bước 2 – Cài đặt package HTTP (nếu cần)

Ví dụ dùng http:

```
dependencies:
  http: ^0.13.6
```

Bước 3 – Viết hàm retry cơ bản

```
Future<String> fetchDataWithRetry() async {
  const int maxAttempts = 3;
  int attempt = 0;

  while (attempt < maxAttempts) {
    try {
      attempt++;

      final response = await http.get(
        Uri.parse("https://api.example.com/data"),
      );

      if (response.statusCode == 200) {
        return response.body;
      } else if (response.statusCode >= 500) {
        throw Exception("Server error");
      } else {
        // Lỗi không nên retry
        throw Exception("Client error");
      }

    } catch (e) {
      if (attempt >= maxAttempts) {
        rethrow;
      }

      await Future.delayed(Duration(seconds: 2));
    }
  }
```


PhucPT10@fpt.edu.vn

```
  throw Exception("Failed after retry");
}
```

🔍 Giải thích syntax quan trọng

`while (attempt < maxAttempts)`
→ Giới hạn số lần retry

`await Future.delayed()`
→ Delay giữa các lần

`rethrow`
→ Ném lỗi ra ngoài sau khi hết retry

⚠ Lưu ý quan trọng

❌ Không dùng `while(true)`
❌ Không retry tất cả lỗi
❌ Không bỏ delay

📌 Ý nghĩa

Giúp app:
Không crash khi mạng chập chờn
Không spam server

PHẦN 2 – CACHING CƠ BẢN

🎯 Mục tiêu kỹ thuật

Lưu dữ liệu API vào local
Hiển thị cache trước
Refresh sau

🧠 Bước 1 – Hiểu chiến lược đơn giản

Flow:
Đọc cache
Nếu có → hiển thị
Gọi API
Lưu lại cache mới

🛠 Bước 2 – Lưu cache bằng Hive

Package:
Hive

🛠 Bước 3 – Khởi tạo Hive

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  await Hive.openBox("productBox");
  runApp(MyApp());
}
```


PhucPT10@fpt.edu.vn

🛠 Bước 4 – Lưu cache

```
final box = Hive.box("productBox");

await box.put("product_list", productList);
```

🛠 Bước 5 – Đọc cache

```
final cachedData = box.get("product_list");
```

🔍 Giải thích

`put(key, value)` → Lưu dữ liệu
`get(key)` → Lấy dữ liệu
Key phải cố định

⚠ Lưu ý

❌ Không cache dữ liệu nhạy cảm
❌ Không quên await
❌ Không đọc trong build()

📌 Ý nghĩa

App mở nhanh hơn
Offline vẫn dùng được

PhucPT10@fpt.edu.vn

HIỂU ĐÚNG TRƯỚC KHI LÀM CRUD

⚠ SharedPreferences:

Là **key–value storage**
Không phải database
Không có bảng, không có query
Không có WHERE, không có filter

👉 Vì vậy "CRUD" ở đây nghĩa là:

CRUD	Trong SharedPreferences
Create	set value
Read	get value
Update	set lại value
Delete	remove

1 BƯỚC 1 – CÀI ĐẶT PACKAGE

✅ Thêm vào pubspec.yaml

```
dependencies:
  shared_preferences: ^2.2.2
```

✅ Import

```
import 'package:shared_preferences/shared_preferences.dart';
```

2 BƯỚC 2 – LẤY INSTANCE (QUAN TRỌNG NHẤT)

SharedPreferences hoạt động bất đồng bộ.

✅ Syntax chuẩn

```
final prefs = await SharedPreferences.getInstance();
```

🔎 Ý nghĩa

Mỗi lần thao tác phải có `prefs`
Phải dùng `await`
Không dùng trong build()

3 CREATE – LƯU DỮ LIỆU

3.1 Lưu String

```
await prefs.setString('token', 'abc123');
```

✓ Giải thích

'token' → key
'abc123' → value
setString trả về bool

3.2 Lưu bool

```
await prefs.setBool('isLoggedIn', true);
```

3.3 Lưu int

```
await prefs.setInt('age', 20);
```

3.4 Lưu List<String>

```
await prefs.s etStringList('tags', ['flutter', 'dart']);
```


PhucPT10@fpt.edu.vn

⚠ LƯU Ý QUAN TRỌNG

❌ Không lưu object trực tiếp

Sai:

```
await prefs.setString('user', userObject);
```

✓ Phải encode JSON:

```
import 'dart:convert';

String userJson = jsonEncode(user.toJson());
await prefs.setString('user', userJson);
```

4 READ – ĐỌC DỮ LIỆU

4.1 Đọc String

```
String? token = prefs.getString('token');
```

4.2 Đọc bool

```
bool isLoggedIn = prefs.getBool('isLoggedIn') ?? false;
```

✓ Vì sao dùng ??

Nếu chưa tồn tại → trả null

?? giúp gán giá trị mặc định

4.3 Đọc List

```
List<String>? tags = prefs.getStringList('tags');
```

4.4 Đọc JSON object

```
String? userJson = prefs.getString('user');

if (userJson != null) {
  Map<String, dynamic> map = jsonDecode(userJson);
  User user = User.fromJson(map);
}
```

⚠ LỖI THƯỜNG GẶP

❌ Quên await khi getInstance

❌ Đọc trong build()

Sai:

```
@override
Widget build(BuildContext context) {
  final prefs = await SharedPreferences.getInstance(); // ❌
}
```

✓ Đúng:

```
@override
void initState() {
  super.initState();
  loadData();
}

Future<void> loadData() async {
  final prefs = await SharedPreferences.getInstance();
}
```


PhucPT10@fpt.edu.vn

5 UPDATE – CẬP NHẬT DỮ LIỆU

SharedPreferences không có update riêng.

👉 Update = set lại value với cùng key.

Ví dụ

```
await prefs.setString('token', 'new_token_456');
```

Key cũ sẽ bị ghi đè.

Ví dụ cập nhật JSON object

```
User updatedUser = user.copyWith(name: 'New Name');

await prefs.setString(
  'user',
  jsonEncode(updatedUser.toJson()),
);
```

6 DELETE – XÓA DỮ LIỆU

Xóa 1 key

```
await prefs.remove('token');
```

Xóa toàn bộ

```
await prefs.clear();
```

⚠ Cẩn thận vì clear() sẽ xóa tất cả key.

7 TỔ CHỨC CODE ĐÚNG (BEST PRACTICE)

Không nên gọi SharedPreferences trực tiếp trong UI.

Tạo service riêng

```
class LocalStorageService {
  Future<void> saveToken(String token) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('token', token);
  }

  Future<String?> getToken() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString('token');
  }

  Future<void> clearToken() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove('token');
  }
}
```

Ý nghĩa

UI không biết chi tiết storage
Dễ thay bằng Secure Storage sau này
Dễ test

PhucPT10@fpt.edu.vn

8 TEST THỰC HÀNH STEP-BY-STEP

Sinh viên nên tự thực hiện:

Bài 1 – Lưu & đọc theme

Tạo nút đổi theme
Khi bấm → setBool('darkMode', true/false)
Khi mở app → đọc value
Apply theme tương ứng

Bài 2 – Login giả lập

Nhập username/password
Nếu đúng → setBool('isLoggedIn', true)
Khi mở app → kiểm tra isLoggedIn
Nếu true → skip màn login
Logout → remove key

PhucPT10@fpt.edu.vn

PHẦN 4 HƯỚNG DẪN CHI TIẾT: HIVE (Local NoSQL Database trong Flutter)

Mục tiêu: Sinh viên **hiểu bản chất Hive**, biết **khi nào dùng**, và có thể **triển khai từng bước** lưu – đọc – cập nhật – xoá dữ liệu local.

I. Hive Là Gì? (Hiểu Đúng Trước Khi Code)

◈ Bản chất

Hive là **NoSQL database thuần Dart**
Lưu dữ liệu dưới dạng **key–value**
Không cần SQLite engine
Rất nhanh, phù hợp mobile

◈ So với SQLite

Hive	SQLite
Key–Value	Table–Row
Không cần SQL	Phải viết query
Nhanh, nhẹ	Chuẩn quan hệ
Tốt cho app nhỏ–vừa	Tốt cho dữ liệu phức tạp

II. Khi Nào Dùng Hive?

✓ Phù hợp:

App Todo offline
Lưu giỏ hàng
Lưu danh sách sản phẩm cache
Lưu cấu hình người dùng
App cần tốc độ cao

✕ Không phù hợp:

Hệ thống cần quan hệ phức tạp nhiều bảng
Dữ liệu rất lớn và phức tạp

III. QUY TRÌNH TRIỂN KHAI HIVE – STEP BY STEP

◈ STEP 1 – Cài đặt package

Thêm vào `pubspec.yaml`

```
dependencies:
  hive: ^2.2.3
  hive_flutter: ^1.1.0
```

Nếu lưu object phức tạp:

```
dev_dependencies:
  hive_generator: ^2.0.0
  build_runner: ^2.4.0
```

Sau đó chạy:

```
flutter pub get
```

◈ STEP 2 – Khởi tạo Hive

Trong `main.dart`:

```
import 'package:hive_flutter/hive_flutter.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Hive.initFlutter();
```


PhucPT10@fpt.edu.vn

```
  runApp(MyApp());
}
```

🎯 Ý nghĩa

Hive cần khởi tạo trước khi mở box
Nếu quên → lỗi runtime

🔷 STEP 3 – Mở Box (Giống như Table)

Box = nơi lưu dữ liệu

```
await Hive.openBox('todoBox');
```

Hoặc:

```
var box = Hive.box('todoBox');
```

🎯 Lưu ý

Phải `openBox()` trước khi dùng
Mỗi box giống một file local riêng

🔷 STEP 4 – Lưu Dữ Liệu (Insert)

📌 Lưu đơn giản

```
var box = Hive.box('todoBox');

box.put('task1', 'Learn Flutter');
```

🎯 Giải thích

`'task1'` là key
`'Learn Flutter'` là value

📌 Lưu danh sách

```
box.put('tasks', ['A', 'B', 'C']);
```

🔷 STEP 5 – Đọc Dữ Liệu

```
var task = box.get('task1');
print(task);
```

Có thể thêm default:

```
var task = box.get('task1', defaultValue: 'No Task');
```

🔷 STEP 6 – Cập Nhật

```
box.put('task1', 'Learn Hive');
```

Hive sẽ ghi đè key cũ.

🔷 STEP 7 – Xoá

```
box.delete('task1');
```

Xoá toàn bộ box:

```
box.clear();
```

IV. Lưu Object (Quan Trọng Nhất)

Nếu lưu model như:

```
class Todo {
  String title;
  bool isDone;
}
```

Ta phải tạo Adapter.

PhucPT10@fpt.edu.vn

◈ STEP 1 – Annotate Model

```
import 'package:hive/hive.dart';

part 'todo.g.dart';

@HiveType(typeId: 0)
class Todo {

  @HiveField(0)
  final String title;

  @HiveField(1)
  final bool isDone;

  Todo({required this.title, required this.isDone});
}
```

◈ STEP 2 – Generate Code

```
flutter pub run build_runner build
```

◈ STEP 3 – Register Adapter

Trong `main()`:

```
Hive.registerAdapter(TodoAdapter());
```

◈ STEP 4 – Lưu Object

```
var box = Hive.box<Todo>('todoBox');

box.add(Todo(title: "Study Hive", isDone: false));
```

◈ STEP 5 – Đọc List Object

```
var todos = box.values.toList();
```

V. ỨNG DỤNG THỰC TẾ

◈ Ví dụ 1 – Todo App Offline

Nếu không có Hive:

- Mở app lại → mất hết task
- Mất mạng → không dùng được

Có Hive:

- Task lưu local
- App mở lại vẫn còn
- Có thể sync server sau

◈ Ví dụ 2 – Cache Product List

Luồng:

- Gọi API
- Lưu kết quả vào Hive
- Nếu mất mạng → đọc Hive thay API

VI. HIVE + REACTIVE UI

Hive hỗ trợ lắng nghe thay đổi:

```
ValueListenableBuilder(
  valueListenable: Hive.box('todoBox').listenable(),
  builder: (context, box, _) {
    return ListView(
      children: box.values
          .map((e) => Text(e.toString()))
          .toList(),
    );
  },
);
```

🎯 Khi dữ liệu thay đổi → UI tự cập nhật

VII. LỖI PHỔ BIẾN SINH VIÊN HAY MẮC

❌ 1. Quên initFlutter

→ App crash

❌ 2. Không register Adapter

→ Cannot read/write object

❌ 3. Thay đổi model nhưng không regenerate

→ type mismatch

❌ 4. Không đóng box khi cần

```
await Hive.close();
```


PhucPT10@fpt.edu.vn

PHẦN 5 HƯỚNG DẪN CHI TIẾT SQLITE

Mục tiêu: Sinh viên hiểu **bản chất SQLite**, biết khi nào dùng, và triển khai **Create – Read – Update – Delete** một cách rõ ràng, có tư duy kiến trúc.
Chúng ta sẽ dùng package chính thức phổ biến nhất trên pub.dev:
👉 sqflite

I. SQLite Là Gì? (Hiểu Bản Chất Trước)

◈ SQLite là gì?

Là **database quan hệ (Relational Database)**
Lưu dữ liệu dạng **table – row – column**
Dùng ngôn ngữ **SQL**

◈ Khác Hive ở đâu?

SQLite	Hive
Có table & schema	Key-value
Có quan hệ	Không quan hệ
Query bằng SQL	Không cần SQL
Phù hợp hệ thống phức tạp	Phù hợp app nhỏ – vừa

II. Khi Nào Nên Dùng SQLite?

✓ App có nhiều bảng liên quan
✓ App có tìm kiếm, filter phức tạp
✓ App bán hàng, quản lý dữ liệu lớn
✓ Offline-first app
❌ Không nên dùng nếu chỉ lưu:
Token
Theme
Cấu hình đơn giản

III. QUY TRÌNH TRIỂN KHAI SQLITE – STEP BY STEP

◈ STEP 1 – Cài Package

```
dependencies:
  sqflite: ^2.3.0
  path: ^1.8.0
```

Sau đó:

```
flutter pub get
```

◈ STEP 2 – Tạo Database Helper (Quan Trọng)

Ta không nên gọi SQL trực tiếp trong UI.
Tạo file:

```
database_helper.dart
```

◈ STEP 3 – Khởi Tạo Database

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {

  static final DatabaseHelper instance = DatabaseHelper._init();

  static Database? _database;
```


PhucPT10@fpt.edu.vn

```
  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('todo.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(
      path,
      version: 1,
      onCreate: _createDB,
    );
  }

  Future _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE todos(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        isDone INTEGER NOT NULL
      )
    ''');
  }
}
```

Giải thích quan trọng

`INTEGER PRIMARY KEY AUTOINCREMENT` → tự tăng id
SQLite không có boolean → dùng INTEGER (0/1)
`onCreate` chỉ ch ạy lần đầu

IV. CRUD – CREATE

Thêm dữ liệu

```
Future<int> insertTodo(String title) async {
  final db = await instance.database;

  return await db.insert(
    'todos',
    {
      'title': title,
      'isDone': 0,
    },
  );
}
```

Giải thích

`db.insert()` an toàn hơn viết raw SQL
Trả về id vừa insert

PhucPT10@fpt.edu.vn

V. READ (Lấy Dữ Liệu)

◈ Lấy tất cả

```
Future<List<Map<String, dynamic>>> getAllTodos() async {
  final db = await instance.database;

  return await db.query('todos');
}
```

◈ Lấy theo điều kiện

```
Future<List<Map<String, dynamic>>> getCompletedTodos() async {
  final db = await instance.database;

  return await db.query(
    'todos',
    where: 'isDone = ?',
    whereArgs: [1],
  );
}
```

🎯 Vì sao dùng ?

Tránh SQL injection
Không viết:

```
where: "isDone = $value"
```

VI. UPDATE

◈ Cập nhật trạng thái

```
Future<int> updateTodo(int id, bool isDone) async {
  final db = await instance.database;

  return await db.update(
    'todos',
    {'isDone': isDone ? 1 : 0},
    where: 'id = ?',
    whereArgs: [id],
  );
}
```

VII. DELETE

◈ Xoá theo id

```
Future<int> deleteTodo(int id) async {
  final db = await instance.database;

  return await db.delete(
    'todos',
    where: 'id = ?',
    whereArgs: [id],
  );
}
```


PhucPT10@fpt.edu.vn

◈ Xoá toàn bộ

```
await db.delete('todos');
```

VIII. DÙNG RAW SQL (Khi Cần)

Insert bằng rawQuery

```
await db.rawInsert(
  'INSERT INTO todos(title, isDone) VALUES(?, ?)',
  ['Learn SQLite', 0],
);
```

Query bằng rawQuery

```
final result = await db.rawQuery(
  'SELECT * FROM todos WHERE isDone = ?',
  [0],
);
```

IX. Chuyển Map → Model (Tư Duy Tốt Hơn)

Tạo model:

```
class Todo {
  final int? id;
  final String title;
  final bool isDone;

  Todo({this.id, required this.title, required this.isDone});

  factory Todo.fromMap(Map<String, dynamic> map) {
    return Todo(
      id: map['id'],
      title: map['title'],
      isDone: map['isDone'] == 1,
    );
  }

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'title': title,
      'isDone': isDone ? 1 : 0,
    };
  }
}
```

X. App Sẽ Tệ Thế Nào Nếu Không Có SQLite?

Ví dụ app Todo:

❌ Tắt app → mất hết dữ liệu

❌ Mất mạng → không dùng được

❌ Dữ liệu lớn → SharedPreferences không chịu nổi

SQLite giúp:

✓ Dữ liệu lưu bền vững

✓ Tìm kiếm nhanh

✓ Không phụ thuộc mạng

PhucPT10@fpt.edu.vn

XI. LỖI SINH VIÊN HAY MẮC

❌ 1. Không await database

→ lỗi chưa mở DB

❌ 2. Không dùng whereArgs

→ SQL injection

❌ 3. Quên convert boolean

→ dữ liệu sai

❌ 4. Không quản lý version database

XII. Nâng Cao – Version & Migration

Nếu thay đổi cấu trúc bảng:

```
openDatabase(
  path,
  version: 2,
  onUpgrade: (db, oldVersion, newVersion) async {
    if (oldVersion < 2) {
      await db.execute('ALTER TABLE todos ADD COLUMN createdAt TEXT');
    }
  },
);
```

XIII. So Sánh Tổng Thể

	SharedPreferences	Hive	SQLite	
Token	✓	✓	✓	
Todo đơn giản	❌	✓	✓	
Quan hệ nhiều bảng	❌	❌	✓	
Query phức tạp	❌	❌	✓	

PhucPT10@fpt.edu.vn

PHẦN 6 – DRIFT trong Flutter (ORM)

I. DRIFT LÀ GÌ? (Hiểu đúng bản chất)

Drift là:

ORM cho SQLite trong Flutter
Viết bằng Dart
Tự generate code
Kiểm tra query compile-time
Hỗ trợ join, relationship
Có migration

👉 Drift = SQLite + type-safe + codegen
👉 Rất gần với Room trên Android

II. Khi nào nên dùng Drift?

✓ App nhiều bảng
✓ Có relationship
✓ Cần join
✓ Offline-first
✓ App enterprise
❌ Không cần nếu chỉ lưu token hoặc vài cấu hình nhỏ

III. CÀI ĐẶT DRIFT – STEP BY STEP

◈ STEP 1 – Thêm package

```
dependencies:
  drift: ^2.0.0
  drift_flutter: ^2.0.0

dev_dependencies:
  drift_dev: ^2.0.0
  build_runner: ^2.4.0
```

Sau đó:

```
flutter pub get
```

IV. TẠO DATABASE CLASS

Tạo file:

```
app_database.dart
```

◈ STEP 2 – Khai báo table

Ví dụ Todo table:

```
import 'package:drift/drift.dart';

class Todos extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get title => text()();
  BoolColumn get isDone => boolean().withDefault(Constant(false))();
}
```

🎯 Điều quan trọng

Không viết SQL string
Drift dùng DSL type-safe
Compile-time check

PhucPT10@fpt.edu.vn

◈ STEP 3 – Tạo Database

```
import 'package:drift/drift.dart';
import 'package:drift_flutter/drift_flutter.dart';

part 'app_database.g.dart';

@DriftDatabase(tables: [Todos])
class AppDatabase extends _$AppDatabase {

  AppDatabase() : super(driftDatabase(name: 'app.db'));

  @override
  int get schemaVersion => 1;
}
```

◈ STEP 4 – Generate code

```
flutter pub run build_runner build
```

Drift sẽ tạo file `.g.dart`

V. CRUD TRONG DRIFT

1 CREATE (Insert)

```
Future<int> insertTodo(String title) {
  return into(todos).insert(
    TodosCompanion(
      title: Value(title),
    ),
  );
}
```

🎯 `TodosCompanion` giúp quản lý nullable khi insert.

2 READ

Lấy tất cả

```
Future<List<Todo>> getAllTodos() {
  return select(todos).get();
}
```

Stream reactive (điểm mạnh)

```
Stream<List<Todo>> watchTodos() {
  return select(todos).watch();
}
```

👉 Khi DB thay đổi → UI tự update
Giống Room + LiveData.

3 UPDATE

```
Future updateTodo(Todo todo) {
  return update(todos).replace(todo);
}
```


PhucPT10@fpt.edu.vn

4 DELETE

```
Future deleteTodo(int id) {
  return (delete(todos)..where((t) => t.id.equals(id))).go();
}
```

VI. JOIN TABLE (Điểm Hive không làm được)

Ví dụ thêm bảng Users

```
class Users extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get name => text()();
}
```

Thêm foreign key vào Todos:

```
IntColumn get userId => integer().references(Users, #id)();
```

Join query

```
Future<List<TodoWithUser>> getTodosWithUser() {
  final query = select(todos).join([
    innerJoin(users, users.id.equalsExp(todos.userId))
  ]);

  return query.map((row) {
    return TodoWithUser(
      row.readTable(todos),
      row.readTable(users),
    );
  }).get();
}
```

👉 Đây là điểm khác biệt lớn với Hive.

VII. MIGRATION (CỰC QUAN TRỌNG)

Khi tăng version:

```
@override
int get schemaVersion => 2;
```

Thêm migration:

```
@override
MigrationStrategy get migration => MigrationStrategy(
  onUpgrade: (m, from, to) async {
    if (from < 2) {
      await m.addColumn(todos, todos.createdAt);
    }
  },
);
```

👉 Drift quản lý migration tốt hơn sqflite thuần.

VIII. So sánh Drift vs Hive vs sqflite

	Hive	sqflite	Drift
SQL	❌	✓	✓
Type-safe	✓	❌	✓✓
Join	❌	✓	✓✓
Compile check	❌	❌	✓
Migration	Hạn chế	Thủ công	Mạnh

PhucPT10@fpt.edu.vn

IX. Drift tương đương gì bên Android/iOS?

Flutter	Android	iOS
Drift	Room	Core Data

Room và Drift gần nhau về:

Annotation
Codegen
Migration
Join
Reactive stream

X. LỖI SINH VIÊN HAY MẮC

❌ Không chạy build_runner
❌ Sửa table nhưng không regenerate
❌ Quên tăng schemaVersion
❌ Không hiểu Companion

XI. Kiến trúc chuẩn khi dùng Drift

```
UI
↓
Repository
↓
Drift DAO
↓
SQLite
```

Không bao giờ gọi DB trực tiếp từ UI.

XII. Khi nào nên dạy Drift cho sinh viên?

🟢 Khi sinh viên đã hiểu:
SQLite
CRUD
async
architecture cơ bản

🔴 Không nên dạy cho người mới hoàn toàn.

XIII. Tóm lại trong 1 câu

Drift là ORM mạnh mẽ, type-safe cho SQLite trong Flutter và là giải pháp tương đương Room trên Android.

PHẦN 7 – 1 SO SÁNH TỔNG QUAN – ÁNH XẠ CÔNG NGHỆ

Mục đích	Flutter	Android (Java/Kotlin)	iOS (Swift)
Lưu key–value nhỏ	SharedPreferences	SharedPreferences	UserDefaults
SQLite thuần	sqflite	SQLiteOpenHelper	SQLite3
ORM trên SQLite	drift	Room	Core Data
NoSQL local	Hive	(ít dùng, có Realm)	Realm
Reactive DB	drift + Stream	Room + LiveData / Flow	Core Data + Combine
Quan sát dữ liệu thay đổi	Stream	LiveData / Flow	Combine / NotificationCenter
Chia sẻ dữ liệu giữa app	(hiếm dùng)	ContentProvider	App Groups / Core Data

PhucPT10@fpt.edu.vn

2 SO SÁNH CHI TIẾT TỪNG KỸ THUẬT

1. SharedPreferences

Flutter

Package: `shared_preferences`
Lưu key–value
Async API

Android

`SharedPreferences`
Lưu key–value XML
Dùng `apply()` hoặc `commit()`

iOS

`UserDefaults`
Lưu key–value
Dữ liệu nhỏ

Mục đích chung

Lưu token
Lưu flag login
Lưu theme
Lưu setting nhỏ

So sánh bản chất

Đặc điểm	Flutter	Android	iOS
Lưu object lớn	❌	❌	❌
Query phức tạp	❌	❌	❌
Phù hợp database	❌	❌	❌

👉 Tất cả đều giống nhau về triết lý:
Chỉ dùng cho dữ liệu nhỏ.

2. SQLite (sqflite)

Flutter

Package: `sqflite`
Viết raw SQL
Trả về Future

Android

`SQLiteOpenHelper`
Viết SQL string
Cursor để đọc dữ liệu

iOS

SQLite3 C API
Viết SQL thủ công

Mục đích chung

Lưu danh sách lớn
Query có WHERE
Filter / sort

PhucPT10@fpt.edu.vn

🔎 So sánh bản chất

Đặc điểm	Flutter sqflite	Android SQLite	iOS SQLite
Raw SQL	Có	Có	Có
Type safety	Thấp	Thấp	Thấp
Reactive	Không	Không	Không
Dễ sai tên cột	Có	Có	Có

👉 Đây là mức thấp nhất của local database.

◈ 3. Drift (Flutter) ↔ Room (Android) ↔ Core Data (iOS)

Đây là phần quan trọng nhất.

Flutter – Drift

SQLite bên dưới
Code generation
Type-safe query
Reactive bằng Stream
Hỗ trợ JOIN, GROUP BY

Android – Room

ORM trên SQLite
Annotation @Entity, @Dao
Type-safe
Tích hợp LiveData / Flow

Ví dụ:

```
@Query("SELECT * FROM user")
LiveData<List<User>> getAllUsers();
```

iOS – Core Data

ORM phức tạp
Entity, Managed Object
Tích hợp Combine
Reactive update UI

🔎 So sánh triết lý

Tính năng	Drift	Room	Core Data
ORM	Có	Có	Có
Type safety	Cao	Cao	Cao
Code generation	Có	Có	Có
Reactive	Stream	LiveData / Flow	Combine
Production-ready	Có	Có	Có

👉 Drift chính là "Room của Flutter".

◈ 4. Hive ↔ (Android Realm) ↔ (iOS Realm)

Hive là:

NoSQL
Không cần SQL
Lưu object trực tiếp

Android & iOS thường dùng:

Realm (cross-platform)

PhucPT10@fpt.edu.vn

So sánh bản chất

Đặc điểm	Hive	Realm
NoSQL	Có	Có
Lưu object trực tiếp	Có	Có
Không cần SQL	Có	Có
Dễ dùng	Cao	Cao
Query phức tạp	Hạn chế	Trung bình

👉 Hive giống một giải pháp MVP / startup nhanh.

◈ 5. Stream ↔ LiveData ↔ Flow ↔ Combine

Đây là phần kiến trúc reactive.

Flutter

Stream
StreamBuilder
Drift watch()

Android

LiveData
Kotlin Flow
Observer pattern

iOS

Combine
Publisher / Subscriber

So sánh reactive data

Flutter	Android	iOS
Stream	LiveData	Combine
StreamBuilder	Observer	SwiftUI binding
watch()	observe()	publisher

🎯 Bản chất chung

Khi DB thay đổi:
→ UI tự động cập nhật.
Không cần reload thủ công.

3 SO SÁNH THEO CẤP ĐỘ CHUYÊN NGHIỆP

Mức độ	Flutter	Android	iOS
Cơ bản	SharedPreferences	SharedPreferences	UserDefaults
Trung bình	sqflite / Hive	SQLite	SQLite
Production	Drift	Room	Core Data
Reactive nâng cao	Drift + Stream	Room + Flow	Core Data + Combine

4 TƯ DUY KIẾN TRÚC (QUAN TRỌNG)

🧠 Flutter Drift + Stream ≈ Android Room + LiveData

Triết lý giống nhau:
Tách Data Layer
ORM thay raw SQL

PhucPT10@fpt.edu.vn

Reactive update UI
Không query DB trong UI

🧠 Flutter SharedPreferences ≈ Android SharedPreferences ≈ iOS UserDefaults

Triết lý giống nhau:
Lưu config
Không phải database

5 MỨC ĐỘ TIẾN HÓA CỦA SINH VIÊN

Nếu sinh viên chỉ biết:
SharedPreferences
sqflite
→ Mới mức demo.
Nếu sinh viên biết:
Drift
Stream
Repository pattern
→ Tiệm cận Android Room & iOS Core Data.

6 KẾT LUẬN HỌC THUẬT QUAN TRỌNG

Slot 14 thực chất đang dạy:
Cách xây dựng Data Layer chuyên nghiệp tương đương native Android & iOS.
Flutter không đơn giản hơn native.
Nó chỉ:
Trừu tượng hóa (abstract)
Đơn giản hóa cú pháp
Nhưng kiến trúc thì tương đương

🎯 SAU KHI HIỂU SO SÁNH NÀY

Sinh viên phải nhận ra:
Drift ≈ Room
Stream ≈ LiveData / Combine
Hive ≈ Realm
SharedPreferences ≈ UserDefaults
Và quan trọng nhất:
Kiến trúc data tốt là universal — không phụ thuộc framework.

PHẦN 8 – ASYNC & LOCAL STORAGE

🎯 Hiểu bản chất

Local storage:
Truy cập disk
Chậm hơn RAM
Phải async

🛠 Ví dụ sai

```
String token = getToken(); // Sai
```

🛠 Ví dụ đúng

```
String? token = await getToken();
```


PhucPT10@fpt.edu.vn

⚠ Nếu không await?

UI hiển thị null
Logic sai
Bug ngầm

📌 TỔNG QUY TRÌNH ÁP DỤNG SLOT 14

Khi gọi API:

Thêm retry có giới hạn
Nếu thành công → cache
Nếu lỗi → dùng cache fallback

Khi login:

Gọi API
Nhận token
Lưu SharedPreferences
Mở app → đọc token

Khi lưu danh sách:

Insert vào DB
await
Load lại UI

🎯 SAU KHI HỌC XONG PHẦN NÀY

Sinh viên phải:

Viết retry an toàn
Lưu token đúng cách
Cache dữ liệu API
CRUD với SQLite/Hive
Không block UI
Không phụ thuộc hoàn toàn network

PHẦN G. TỔNG KẾT & ĐỊNH HƯỚNG

◈ Vai trò của Retry & Persistence

Là nền tảng của mobile architecture
Không có → app không thể production

◈ Chuẩn bị cho các bước tiếp theo

Sau Slot 14, sinh viên sẵn sàng học:

Repository Pattern
Clean Architecture
Offline-first strategy
Sync engine

🎯 KẾT LUẬN CUỐI CÙNG

Sau phần này sinh viên phải:
Hiểu bản chất:

Retry là cơ chế chịu lỗi
Cache là lưu tạm có chiến lược
Persistence là nền tảng của mobile app

Phân biệt rõ:

SharedPreferences → dữ liệu nhỏ

PhucPT10@fpt.edu.vn

Hive → object nhanh, MVP
SQLite → dữ liệu quan hệ

Và quan trọng nhất:

App thật không thể chỉ gọi API rồi hiển thị.

Một mobile app chuyên nghiệp phải:

Sống sót khi mạng yếu
Không mất dữ liệu khi đóng app
Luôn có phương án dự phòng

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. Kỹ thuật: Retry Logic

Kinh nghiệm thực tế

Dùng ở đâu trong app thật?

Gọi API load danh sách
Refresh token
Upload ảnh
Đồng bộ dữ liệu nền

Tình huống thực tế

Bối cảnh:
App thương mại nhỏ, gọi API lấy danh sách sản phẩm.
Vấn đề:
Người dùng ở vùng mạng yếu → request timeout → app báo lỗi ngay → UX rất tệ.
Giải pháp:
Triển khai retry có kiểm soát thay vì fail ngay.
Vì sao chọn retry thay vì chỉ show lỗi?
70% lỗi là lỗi mạng tạm thời
Retry giúp tăng tỉ lệ thành công đáng kể

Lỗi thường gặp

Retry vô hạn

Nguyên nhân:
Không giới hạn số lần retry.
Ví dụ sai:

```
while (true) {
  await apiCall();
}
```

Khắc phục:

```
Future<T> retry<T>(Future<T> Function() task) async {
  int maxRetry = 3;
  for (int i = 0; i < maxRetry; i++) {
    try {
      return await task();
    } catch (_) {
      await Future.delayed(Duration(seconds: 2));
    }
  }
  rethrow;
}
```

Phòng tránh:
Luôn có maxRetry
Có delay giữa các lần retry

Retry cả lỗi 400/401

Sai tư duy:
Retry mọi lỗi.
Thực tế:
400 → sai request
401 → token hết hạn

PhucPT10@fpt.edu.vn

Retry không có ý nghĩa.

💡 Tips thực chiến

Log số lần retry
Chỉ retry lỗi network (SocketException)
Dùng exponential backoff

🛡 Best Practices

Retry có giới hạn
Retry có delay tăng dần
Không retry lỗi logic
Tách retry thành utility riêng

🔄 Case thực tế

App bị crash do retry vô hạn khi server down.
Sau khi:

Thêm maxRetry = 3
Thêm delay

👉 Giảm 80% crash liên quan network.

🎓 Bài học

Retry không phải gọi lại vô hạn.

🔷 2. Kỹ thuật: Caching Strategy

✅ Kinh nghiệm thực tế

📌 Dùng ở đâu?

Load danh sách khi mở app
Trang chủ
Profile

🔷 Tình huống

App load danh sách 2–3 giây mỗi lần mở.
User nghĩ app chậm.
Giải pháp:
Cache local DB + network-first strategy.

⚠ Lỗi thường gặp

❌ Cache không có TTL

🔍 Sai tư duy:
Cache mãi mãi.
🧪 Ví dụ sai:

```
if (cache != null) return cache;
```

🛠 Giải pháp:
Lưu timestamp:

```
{
  "data": [...],
  "cachedAt": 123456789
}
```

Kiểm tra TTL trước khi dùng.

PhucPT10@fpt.edu.vn

❌ Không phân biệt chiến lược cache

Network-first
Cache-first
Stale-while-revalidate

💡 Tips

Trang chủ: cache-first
Thanh toán: network-first
Tin tức: stale-while-revalidate

🛡 Best Practices

Cache có thời hạn
Luôn có fallback
Không cache dữ liệu nhạy cảm

🎓 Bài học

Cache không phải lưu cho nhanh. Cache phải có chiến lược.

🔷 3. SharedPreferences

shared_preferences

✅ Kinh nghiệm thực tế

📌 Dùng để:

Lưu token
Lưu theme
Lưu flag onboarding

🔷 Tình huống

Sinh viên lưu cả danh sách 2000 item vào SharedPreferences.
App lag.

⚠ Lỗi thường gặp

❌ Lưu object lớn

🔍 Nguyên nhân:
Hiểu nhầm SharedPreferences là database.

🧪 Sai:

```
prefs.setString("bigList", jsonEncode(hugeList));
```

🛠 Đúng:

Chỉ lưu primitive nhỏ
Dùng Hive/SQLite cho list lớn

❌ Không await

```
prefs.setBool("isLogin", true);
```

Không await → race condition.

💡 Tips

Chỉ lưu cấu hình nhỏ
Tách lớp PreferencesService

PhucPT10@fpt.edu.vn

🛡 Best Practices

Không lưu password plain text
Không dùng cho list lớn
Luôn await

🎓 Bài học

SharedPreferences không phải database.

4. SQLite (sqflite)

sqflite

✅ Kinh nghiệm thực tế

📌 Dùng khi:

Danh sách lớn
Tìm kiếm
Quan hệ nhiều bảng

🔷 Tình huống

App lưu 5000 item bằng SharedPreferences → crash.
Chuyển sang SQLite → ổn định.

⚠ Lỗi thường gặp

❌ Gọi DB trong build()

```
@override
Widget build(BuildContext context) {
  database.getAll(); // sai
}
```

Gây loop rebuild.

❌ Không await DB

UI render trước khi dữ liệu xong.

💡 Tips

Tạo singleton DatabaseService
Không truy cập DB trực tiếp từ UI

🛡 Best Practices

Tách repository layer
Migration version rõ ràng
Không block main isolate

🎓 Bài học

DB không bao giờ được gọi trong build().

5. Hive

hive

✅ Kinh nghiệm thực tế

Phù hợp:

App offline

PhucPT10@fpt.edu.vn

Dữ liệu dạng object
Không cần join phức tạp

◈ Tình huống
App cần load nhanh khi offline.
Hive load nhanh hơn SQLite cho object đơn giản.

⚠ Lỗi thường gặp

✕ Không close box
Memory leak.

✕ Không xử lý migration
Thay đổi model → crash.

💡 Tips
Mở box một lần khi app start
Dùng TypeAdapter rõ ràng

🛡 Best Practices
Không dùng cho quan hệ phức tạp
Không lưu dữ liệu cực lớn

🎓 Bài học
Hive nhanh, nhưng không thay thế SQL khi cần quan hệ.

◈ 6. drift
drift

☑ Kinh nghiệm thực tế
Dùng khi:
Cần type safety
Cần reactive query
App trung bình → lớn

◈ Tình huống
App cần auto refresh khi DB thay đổi.
Drift cung cấp:
```
select(todos).watch();
```
UI tự update.

⚠ Lỗi thường gặp

✕ Không dùng Stream mà dùng Future
Mất reactive.

✕ Không cấu hình isolate
Query lớn gây lag.

💡 Tips
Luôn dùng watch() cho UI

PhucPT10@fpt.edu.vn

Tách database file riêng

🛡 Best Practices

Drift cho production
Dùng StreamBuilder đúng cách
Không gọi query trong initState nhiều lần

🎓 Bài học

Drift là Room của Flutter.

7. Stream

✅ Kinh nghiệm thực tế

Dùng để:

Reactive UI
Lắng nghe DB thay đổi

⚠ Lỗi phổ biến

❌ Không dispose stream

Memory leak.

❌ Gọi setState sau dispose

Crash.

💡 Tips

Dùng StreamBuilder thay vì manual subscription
Kiểm tra mounted trước setState

🎓 Bài học

Stream mạnh, nhưng phải quản lý lifecycle.

8. Async/Await với Storage

⚠ Lỗi phổ biến

❌ Không await

Race condition.

❌ Gọi async trong build

Loop rebuild.

🛡 Best Practices

Async chỉ gọi ở:
- initState
- service layer

Không block UI

9. Thiết kế Data Layer

PhucPT10@fpt.edu.vn

📌 Production structure

```
data/
  ├── models/
  ├── services/
  ├── repositories/
  ├── local/
  └── remote/
```

❌ Sai phổ biến

UI gọi DB trực tiếp.

🛡 Best Practices

UI → Repository → Local/Remote
Không bao giờ:
UI → SQLite trực tiếp

🎯 TỔNG KẾT CHO SINH VIÊN

Nếu chỉ nhớ 5 điều:

Retry phải có giới hạn
Cache phải có chiến lược
SharedPreferences không phải DB
Không gọi DB trong build()
Tách data layer khỏi UI

🎓 Thông điệp quan trọng nhất

App tốt không phải app chạy khi mạng hoàn hảo.
App tốt là app vẫn hoạt động khi mạng không hoàn hảo.

PhucPT10@fpt.edu.vn

EXERCISE

📄 BÀI TẬP TỔNG HỢP

SMART TASK MANAGER (Offline-first)

PHẦN 1 – DỰ ÁN CƠ BẢN (SharedPreferences)

1️⃣ TÓM TẮT THÔNG TIN DỰ ÁN

📌 Tên dự án

Smart Task Manager – Basic Persistence

🎯 Mục tiêu học tập (học xong phải làm được)

Hiểu **Local Persistence** là gì (lưu dữ liệu vào máy, không cần mạng).

Biết cách **serialize JSON**: đổi `List<Task>` → `String JSON` để lưu.

Nắm **async/await** khi đọc/ghi storage (đọc ghi là việc "mất thời gian").

Biết **giới hạn** của SharedPreferences: chỉ phù hợp dữ liệu **nhỏ**, không phải database.

2️⃣ 🧩 CHỨC NĂNG (đầy đủ CRUD task)

Create: Thêm task mới

Read: Hiển thị danh sách task (load từ SharedPreferences khi mở app)

Update:

- Sửa tiêu đề task
- Đánh dấu hoàn thành/chưa hoàn thành

Delete: Xoá task

Persistence:

- Lưu danh sách khi thao tác (thêm/sửa/xoá/toggle)
- Mở app lại tự load ra như cũ

3️⃣ 🛠 Công nghệ sử dụng (cơ bản, phù hợp mới học)

Flutter SDK (khuyến nghị Flutter stable)

Dart

IDE: **VS Code** (hoặc Android Studio)

Package:

- `shared_preferences` (lưu key–value)

(Tuỳ chọn) Thiết bị test: Android Emulator / iPhone Simulator / điện thoại thật

4️⃣ 📂 CẤU TRÚC THƯ MỤC

project-structure.txt

```
lib/
 ├── main.dart
 ├── models/
 │    └── task.dart
 ├── services/
 │    └── preferences_service.dart
 ├── screens/
 │    └── home_screen.dart
```

Giải thích dễ hiểu:

PhucPT10@fpt.edu.vn

`models/`: chứa **dữ liệu** (Task)
`services/`: chứa **logic lưu/đọc SharedPreferences**
`screens/`: giao diện màn hình (HomeScreen)
`main.dart`: chạy app, khai báo MaterialApp

5️⃣ 📋 TÓM TẮT DANH SÁCH BƯỚC THỰC HIỆN

1. Tạo project Flutter
2. Thêm dependency shared_preferences
3. Tạo model Task + toJson/fromJson
4. Tạo PreferencesService để **save/load**
5. Tạo UI cơ bản (List + thêm task)
6. Gắn logic lưu & load (persistence)
7. Bổ sung CRUD đầy đủ (sửa, xoá, toggle done)
8. Test và kiểm chứng persistence
9. Tổng hợp lỗi phổ biến + cách tránh

6️⃣ CHI TIẾT TỪNG BƯỚC (đến từng thao tác)

Bước 1: Tạo project

Mở Terminal (trong VS Code cũng được) và chạy:

step1_create_project.sh

```
flutter create smart_task_manager
cd smart_task_manager
code .
```

✅ **Kiểm chứng:**

Chạy thử app mẫu:

step1_run.sh

```
flutter run
```

Nếu hiện app đếm số (counter) → môi trường OK.

Bước 2: Thêm dependency SharedPreferences

Mở file `pubspec.yaml` và thêm:

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  shared_preferences: ^2.2.2
```

Chạy:

step2_pubget.sh

```
flutter pub get
```

✅ **Kiểm chứng:**

Không báo lỗi version conflict.
Trong `pubspec.lock` có `shared_preferences`.

Bước 3: Tạo Model Task

PhucPT10@fpt.edu.vn

Tạo file: `lib/models/task.dart`

lib/models/task.dart

```
class Task {
  String id;
  String title;
  bool isDone;

  Task({
    required this.id,
    required this.title,
    this.isDone = false,
  });

  Map<String, dynamic> toJson() => {
        'id': id,
        'title': title,
        'isDone': isDone,
      };

  factory Task.fromJson(Map<String, dynamic> json) => Task(
        id: json['id'] as String,
        title: json['title'] as String,
        isDone: (json['isDone'] as bool?) ?? false,
      );
}
```

✅ Kiểm chứng (tư duy):
`toJson()` biến Task thành Map → có thể encode thành JSON string.
`fromJson()` tạo Task từ Map → có thể decode từ JSON.

Bước 4: Tạo Preferences Service (save/load bằng JSON)

Tạo file: `lib/services/preferences_service.dart`

lib/services/preferences_service.dart

```
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';
import '../models/task.dart';

class PreferencesService {
  static const String key = "task_list";

  Future<void> saveTasks(List<Task> tasks) async {
    final prefs = await SharedPreferences.getInstance();
    final jsonString = jsonEncode(tasks.map((e) => e.toJson()).toList());
    await prefs.setString(key, jsonString);
  }

  Future<List<Task>> loadTasks() async {
    final prefs = await SharedPreferences.getInstance();
    final jsonString = prefs.getString(key);
    if (jsonString == null) return [];
```

```
    final List decoded = jsonDecode(jsonString) as List;
    return decoded
        .map((e) => Task.fromJson(e as Map<String, dynamic>))
        .toList();
  }

  Future<void> clearTasks() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(key);
  }
}
```

✅ Kiểm chứng kỹ thuật "SharedPreferences lưu được List không?"
SharedPreferences **không lưu trực tiếp List<Task>**.
Ta phải biến nó thành `String` bằng `jsonEncode` → lưu `setString`.

Bước 5: Tạo UI HomeScreen (có thêm/sửa/xoá/toggle)
Tạo file: lib/screens/home_screen.dart
lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';
import '../models/task.dart';
import '../services/preferences_service.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final PreferencesService service = PreferencesService();

  final TextEditingController addController = TextEditingController();

  List<Task> tasks = [];
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    _loadData();
  }

  @override
  void dispose() {
    addController.dispose();
    super.dispose();
  }

  Future<void> _loadData() async {
```


PhucPT10@fpt.edu.vn

```
    final loaded = await service.loadTasks();
    setState(() {
      tasks = loaded;
      isLoading = false;
    });
  }

  Future<void> _saveData() async {
    await service.saveTasks(tasks);
  }

  void _addTask() {
    final title = addController.text.trim();
    if (title.isEmpty) return;

    final task = Task(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      title: title,
    );

    setState(() {
      tasks.add(task);
      addController.clear();
    });

    _saveData();
  }

  void _toggleDone(Task task) {
    setState(() {
      task.isDone = !task.isDone;
    });
    _saveData();
  }

  void _deleteTask(Task task) {
    setState(() {
      tasks.removeWhere((t) => t.id == task.id);
    });
    _saveData();
  }

  Future<void> _editTaskDialog(Task task) async {
    final controller = TextEditingController(text: task.title);

    final result = await showDialog<String>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Sửa task'),
        content: TextField(
          controller: controller,
          autofocus: true,
          decoration: const InputDecoration(
```

```
            hintText: 'Nhập tiêu đề mới',
          ),
          onSubmitted: (value) => Navigator.pop(context, value),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, null),
            child: const Text('Hủy'),
          ),
          ElevatedButton(
            onPressed: () => Navigator.pop(context, controller.text),
            child: const Text('Lưu'),
          ),
        ],
      ),
    );

    if (result == null) return;

    final newTitle = result.trim();
    if (newTitle.isEmpty) return;

    setState(() {
      task.title = newTitle;
    });
    _saveData();
  }

  Future<void> _clearAllDialog() async {
    final ok = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Xóa tất cả?'),
        content: const Text('Bạn có chắc muốn xóa toàn bộ danh sách task
không?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Không'),
          ),
          ElevatedButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Có'),
          ),
        ],
      ),
    );

    if (ok != true) return;

    setState(() {
      tasks.clear();
    });
```


PhucPT10@fpt.edu.vn

```
    await service.clearTasks();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Smart Task Manager (Offline)'),
        actions: [
          IconButton(
            onPressed: _clearAllDialog,
            icon: const Icon(Icons.delete_forever),
            tooltip: 'Xóa tất cả',
          ),
        ],
      ),
      body: isLoading
          ? const Center(child: CircularProgressIndicator())
          : Column(
              children: [
                Padding(
                  padding: const EdgeInsets.all(12),
                  child: Row(
                    children: [
                      Expanded(
                        child: TextField(
                          controller: addController,
                          decoration: const InputDecoration(
                            labelText: 'Nhập task mới',
                            border: OutlineInputBorder(),
                          ),
                          onSubmitted: (_) => _addTask(),
                        ),
                      ),
                      const SizedBox(width: 10),
                      ElevatedButton(
                        onPressed: _addTask,
                        child: const Text('Thêm'),
                      )
                    ],
                  ),
                ),
                const Divider(height: 0),
                Expanded(
                  child: tasks.isEmpty
                      ? const Center(child: Text('Chưa có task nào.'))
                      : ListView.separated(
                          itemCount: tasks.length,
                          separatorBuilder: (_, __) => const Divider(height:
0),
                          itemBuilder: (context, index) {
                            final task = tasks[index];
                            return ListTile(
```


PhucPT10@fpt.edu.vn

```
                                leading: Checkbox(
                                  value: task.isDone,
                                  onChanged: (_) => _toggleDone(task),
                                ),
                                title: Text(
                                  task.title,
                                  style: TextStyle(
                                    decoration: task.isDone
                                        ? TextDecoration.lineThrough
                                        : null,
                                  ),
                                ),
                                onTap: () => _editTaskDialog(task),
                                trailing: IconButton(
                                  icon: const Icon(Icons.delete),
                                  onPressed: () => _deleteTask(task),
                                ),
                              );
                            },
                          ),
                        ),
                      ],
                    ),
    );
  }
}
```

✅ **Kiểm chứng từng kỹ thuật trong UI:**

Thêm: nhập text → bấm "Thêm" → list tăng 1
Toggle done: tick checkbox → chữ gạch ngang
Sửa: bấm vào item → dialog → đổi title → item đổi
Xoá: bấm icon thùng rác → item biến mất
Xoá tất cả: icon `delete_forever` trên AppBar

Bước 6: Nối HomeScreen vào main.dart
Mở/ghi file: lib/main.dart
lib/main.dart

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const SmartTaskManagerApp());
}

class SmartTaskManagerApp extends StatelessWidget {
  const SmartTaskManagerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
```


PhucPT10@fpt.edu.vn

```
      title: 'Smart Task Manager',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.green),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
  }
}
```

✅ Kiểm chứng:
Run app → phải thấy màn HomeScreen.

7️⃣ 🧪 CÁCH KIỂM CHỨNG "PERSISTENCE" (quan trọng nhất)

Test 1: Lưu và tự load lại

Thêm 3 task
Tick hoàn thành 1 task
Tắt app hoàn toàn:
Android emulator: "Close app / swipe away"
Hoặc stop rồi run lại
Mở lại app

✅ Kết quả đúng:
Danh sách task **còn nguyên**
Trạng thái done/undone **còn nguyên**

Test 2: Sửa rồi mở lại

Bấm vào task để sửa title
Tắt app và mở lại

✅ Kết quả đúng: title mới vẫn còn.

8️⃣ ⚠️ LỖI PHỔ BIẾN (và cách né)

Không await khi save/load
Triệu chứng: đôi khi dữ liệu không kịp lưu → mở lại bị mất
Cách né: hàm lưu/đọc luôn là `async` và dùng `await` trong service

Gọi load trong `build()`
Triệu chứng: build chạy nhiều lần → load nhiều lần → lag / vòng lặp
Cách né: gọi trong `initState()`

Lưu dữ liệu quá lớn
SharedPreferences không phải database
Dữ liệu lớn nên chuyển sang SQLite/Isar/Hive (phần sau của Offline-first)

9️⃣ 🎓 BÀI HỌC RÚT RA

SharedPreferences phù hợp để:
config nhỏ, token, theme, settings, list nhỏ đơn giản
Không phù hợp để:
dữ liệu lớn, truy vấn phức tạp, nhiều bảng/quan hệ

1️⃣0️⃣ "CÁC PHIÊN BẢN CẬP NHẬT THEO TỪNG BƯỚC" (theo diễn biến thực hiện)

Dưới đây là cách chia phiên bản để sinh viên dễ theo:

PhucPT10@fpt.edu.vn

Version 0: Chạy app mặc định (Counter)
    Mục tiêu: kiểm tra môi trường
Version 1: Hiển thị HomeScreen + UI nhập task (chưa lưu)
    Có UI thêm task, list hiển thị trong RAM
Version 2: Thêm model Task + hiển thị List<Task>
    Task có id, title, isDone
Version 3: Thêm PreferencesService + lưu/load danh sách (persistence)
    Đóng app mở lại vẫn có task
Version 4: Hoàn thiện CRUD + toggle done + edit dialog + clear all
    Bài hoàn chỉnh như code ở trên

PhucPT10@fpt.edu.vn

PHẦN 2 – SQLITE (sqflite)
SMART TASK MANAGER (Offline-first)

Tài liệu này dành cho sinh viên mới học , hướng dẫn theo kiểu dẫn dắt thực hành từng thao tác , tương tự Phần 1 (SharedPreferences), nhưng thay bằng SQLite (sqflite) .

1️⃣ TÓM TẮT THÔNG TIN DỰ ÁN

📌 Tên dự án
Smart Task Manager – SQLite Persistence

🎯 Mục tiêu học tập (học xong phải làm được)
1. Hiểu SQLite là gì và vì sao nó tốt hơn SharedPreferences khi dữ liệu nhiều.
2. Biết cách tạo Database file trong app, tạo table bằng SQL.
3. Làm được CRUD với sqflite: insert / select / update / delete.
4. Biết cách map dữ liệu giữa:
   - `Task` (Model trong Dart)
   - `Map<String, dynamic>` (dạng DB)
5. Hiểu quy tắc quan trọng:
   - SQLite không có kiểu bool → dùng `INTEGER` (0/1)
   - thao tác DB là async → phải dùng `await`
6. Biết cách kiểm chứng dữ liệu đã lưu b□□□ng: đóng/mở app + log + xem file DB (mức cơ bản)

2️⃣ 🧩 CHỨC NĂNG (đầy đủ CRUD task)
- Create : Thêm task
- Read : Load danh sách task từ SQLite khi mở app
- Update :
  - Toggle done/undone
  - Sửa title
- Delete : Xoá task
- Persistence :
  - Dữ liệu còn nguyên sau khi tắt/mở app
- (Tuỳ chọn nâng cao) Xoá tất cả, sắp xếp theo thời gian tạo

3️⃣ 🛠 Công nghệ sử dụng
- Flutter SDK
- Dart
- VS Code (khuyến nghị)
- Packages:
  - `sqflite` (SQLite cho Flutter)
  - `path` (nối đường dẫn file db cho đúng trên Android/iOS)

4️⃣ 📂 CẤU TRÚC THƯ MỤC (đề xuất dễ học)

PhucPT10@fpt.edu.vn

text
```
lib/
  ├─── main.dart
  ├─── models/
  │     └─── task.dart
  ├─── db/
  │     └─── app_database.dart
  ├─── repositories/
  │     └─── task_repository.dart
  └─── screens/
        └─── home_screen.dart
```

Giải thích:
- `db/app_database.dart`: nơi mở DB + tạo table + quản lý version
- `repositories/task_repository.dart`: nơi viết CRUD (tách khỏi UI cho dễ hiểu)
- `screens/home_screen.dart`: UI gọi repository để thao tác dữ liệu

5️⃣ 📋 TÓM TẮT DANH SÁCH BƯỚC THỰC HIỆN

1. Tạo project Flutter
2. Thêm dependency `sqflite` + `path`
3. Thiết kế bảng `tasks`
4. Tạo `Task` model (toMap/fromMap + toJson/fromJson nếu muốn)
5. Tạo `AppDatabase` (openDatabase, onCreate, version)
6. Tạo `TaskRepository` (CRUD)
7. Tạo UI quản lý task (giống Phần 1 để quen tay)
8. Test persistence + kiểm chứng kỹ thuật
9. Tổng hợp lỗi phổ biến + bài tập thêm

6️⃣ CHI TIẾT TỪNG BƯỚC (dẫn dắt từng thao tác)

Bước 1: Tạo project

Mở Terminal:

bash
```bash
flutter create smart_task_manager_sqlite
cd smart_task_manager_sqlite
code .
```

Chạy thử:

bash
```bash
flutter run
```

✅ Nếu app counter chạy được → OK.

Bước 2: Thêm dependency (sqflite + path)

PhucPT10@fpt.edu.vn

Mở `pubspec.yaml` và thêm:

yaml
```yaml
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.3.3
  path: ^1.9.0
```

Chạy:

bash
```bash
flutter pub get
```

✅ **Kiểm chứng:**
- Không báo lỗi dependency
- Có import được `sqflite` và `path`

Bước 3: Thiết kế Database & Table

3.1 Bảng `tasks` gồm các cột
- `id` (TEXT, primary key)
- `title` (TEXT)
- `is_done` (INTEGER: 0/1)
- `created_at` (INTEGER: millisecondsSinceEpoch)

3.2 Câu lệnh SQL tạo bảng

sql
```sql
CREATE TABLE tasks(
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL,
  is_done INTEGER NOT NULL,
  created_at INTEGER NOT NULL
);
```

✅ Ghi nhớ:
- SQLite không có bool → `is_done` dùng `INTEGER` (0/1)
- `created_at` để sắp xếp (task mới lên đầu)

Bước 4: Tạo Model Task (map <-> db)

Tạo file `lib/models/task.dart`

dart
```dart
class Task {
```


PhucPT10@fpt.edu.vn

```
  final String id;
  String title;
  bool isDone;
  final int createdAt;

  Task({
    required this.id,
    required this.title,
    this.isDone = false,
    required this.createdAt,
  });
```

```
  /// Map dùng cho SQLite (sqflite)
 Map<String, dynamic> toMap() => {
        'id': id,
        'title': title,
        'is_done': isDone ? 1 : 0,
        'created_at': createdAt,
      };

  factory Task.fromMap(Map<String, dynamic> map) => Task(
        id: map['id'] as String,
        title: map['title'] as String,
        isDone: (map['is_done'] as int) == 1,
        createdAt: map['created_at'] as int,
      );
}
```

✅ Kiểm chứng nhanh (tư duy):
- `toMap()` biến Task → Map (DB hiểu được)
- `fromMap()` biến Map → Task (UI hiểu được)

Bước 5: Tạo AppDatabase (mở db + tạo bảng)

Tạo file `lib/db/app_database.dart`

dart

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class AppDatabase {
  AppDatabase._();
  static final AppDatabase instance = AppDatabase._();

  static const String _dbName = 'smart_task_manager.db';
  static const int _dbVersion = 1;

  Database? _db;

  Future<Database> get database async {
```


PhucPT10@fpt.edu.vn

```
    if (_db != null) return _db!;
    _db = await _openDb();
    return _db!;
  }

  Future<Database> _openDb() async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, _dbName);

    return openDatabase(
      path,
      version: _dbVersion,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE tasks(
            id TEXT PRIMARY KEY,
            title TEXT NOT NULL,
            is_done INTEGER NOT NULL,
            created_at INTEGER NOT NULL
          );
        ''');
      },
    );
  }
}
```

✅ Kiểm chứng kỹ thuật:
- `getDatabasesPath()` lấy thư mục chuẩn để lưu db
- `join()` giúp đường dẫn đúng trên nhiều nền tảng
- `openDatabase()` tạo/mở file db
- `onCreate` chỉ chạy lần đầu khi db chưa tồn tại

Bước 6: Tạo Repository (CRUD)

Tạo file `lib/repositories/task_repository.dart`

dart
```
import 'package:sqflite/sqflite.dart';
import '../db/app_database.dart';
import '../models/task.dart';

class TaskRepository {
  static const String table = 'tasks';

  Future<int> insertTask(Task task) async {
    final db = await AppDatabase.instance.database;
    return db.insert(
      table,
      task.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace,
```


PhucPT10@fpt.edu.vn

```
    );
  }

  Future<List<Task>> getAllTasks() async {
    final db = await AppDatabase.instance.database;
    final maps = await db.query(
      table,
      orderBy: 'created_at DESC',
    );
    return maps.map((e) => Task.fromMap(e)).toList();
  }

  Future<int> updateTask(Task task) async {
    final db = await AppDatabase.instance.database;
    return db.update(
      table,
      task.toMap(),
      where: 'id = ?',
      whereArgs: [task.id],
    );
  }

  Future<int> deleteTask(String id) async {
    final db = await AppDatabase.instance.database;
    return db.delete(
      table,
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  Future<int> deleteAll() async {
    final db = await AppDatabase.instance.database;
    return db.delete(table);
  }
}
```

✅ **Kiểm chứng:**
- `insert` trả về số dòng insert (thường là 1)
- `query` trả `List<Map<String, Object?>>`
- `update/delete` trả số dòng bị ảnh hưởng

Bước 7: Tạo UI HomeScreen (thực hành CRUD + persistence)

Tạo file `lib/screens/home_screen.dart`

dart
```
import 'package:flutter/material.dart';
import '../models/task.dart';
import '../repositories/task_repository.dart';
```


PhucPT10@fpt.edu.vn

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final TaskRepository repo = TaskRepository();
  final TextEditingController addController = TextEditingController();

  List<Task> tasks = [];
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    _loadTasks();
  }

  @override
  void dispose() {
    addController.dispose();
    super.dispose();
  }

  Future<void> _loadTasks() async {
    final loaded = await repo.getAllTasks();
    setState(() {
      tasks = loaded;
      isLoading = false;
    });
  }

  Future<void> _addTask() async {
    final title = addController.text.trim();
    if (title.isEmpty) return;

    final task = Task(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      title: title,
      createdAt: DateTime.now().millisecondsSinceEpoch,
    );

    await repo.insertTask(task);
    addController.clear();
    await _loadTasks();
  }

  Future<void> _toggleDone(Task task) async {
    task.isDone = !task.isDone;
    await repo.updateTask(task);
```


PhucPT10@fpt.edu.vn

```
    await _loadTasks();
  }

  Future<void> _deleteTask(Task task) async {
    await repo.deleteTask(task.id);
    await _loadTasks();
  }

  Future<void> _editTaskDialog(Task task) async {
    final controller = TextEditingController(text: task.title);

    final result = await showDialog<String>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Sửa task'),
        content: TextField(
          controller: controller,
          autofocus: true,
          decoration: const InputDecoration(
            hintText: 'Nhập tiêu đề mới',
          ),
          onSubmitted: (value) => Navigator.pop(context, value),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, null),
            child: const Text('Hủy'),
          ),
          ElevatedButton(
            onPressed: () => Navigator.pop(context, controller.text),
            child: const Text('Lưu'),
          ),
        ],
      ),
    );

    if (result == null) return;

    final newTitle = result.trim();
    if (newTitle.isEmpty) return;

    task.title = newTitle;
    await repo.updateTask(task);
    await _loadTasks();
  }

  Future<void> _clearAllDialog() async {
    final ok = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Xóa tất cả?'),
        content: const Text('Bạn có chắc muốn xóa toàn bộ task không?'),
        actions: [
```


PhucPT10@fpt.edu.vn

```
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Không'),
        ),
        ElevatedButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Có'),
        ),
      ],
    ),
  );

  if (ok != true) return;

  await repo.deleteAll();
  await _loadTasks();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Smart Task Manager (SQLite)'),
      actions: [
        IconButton(
          onPressed: _clearAllDialog,
          icon: const Icon(Icons.delete_forever),
          tooltip: 'Xóa tất cả',
        ),
      ],
    ),
    body: isLoading
        ? const Center(child: CircularProgressIndicator())
        : Column(
            children: [
              Padding(
                padding: const EdgeInsets.all(12),
                child: Row(
                  children: [
                    Expanded(
                      child: TextField(
                        controller: addController,
                        decoration: const InputDecoration(
                          labelText: 'Nhập task mới',
                          border: OutlineInputBorder(),
                        ),
                        onSubmitted: (_) => _addTask(),
                      ),
                    ),
                    const SizedBox(width: 10),
                    ElevatedButton(
                      onPressed: _addTask,
                      child: const Text('Thêm'),
```

```
                    )
                  ],
                ),
              ),
              const Divider(height: 0),
              Expanded(
                child: tasks.isEmpty
                    ? const Center(child: Text('Chưa có task nào.'))
                    : ListView.separated(
                        itemCount: tasks.length,
                        separatorBuilder: (_, __) => const Divider(height:
0),
                        itemBuilder: (context, index) {
                          final task = tasks[index];
                          return ListTile(
                            leading: Checkbox(
                              value: task.isDone,
                              onChanged: (_) => _toggleDone(task),
                            ),
                            title: Text(
                              task.title,
                              style: TextStyle(
                                decoration: task.isDone
                                    ? TextDecoration.lineThrough
                                    : null,
                              ),
                            ),
                            subtitle: Text(
                              'createdAt:
${DateTime.fromMillisecondsSinceEpoch(task.createdAt)}',
                              maxLines: 1,
                              overflow: TextOverflow.ellipsis,
                            ),
                            onTap: () => _editTaskDialog(task),
                            trailing: IconButton(
                              icon: const Icon(Icons.delete),
                              onPressed: () => _deleteTask(task),
                            ),
                          );
                        },
                      ),
              ),
            ],
          ),
    );
  }
}
```

✅ Điểm cần chú ý cho sinh viên mới:
- Trong SharedPreferences ta “save list” một lần.
- Trong SQLite ta “lưu từng dòng (row)” bằng insert/update/delete.

PhucPT10@fpt.edu.vn

- UI mỗi lần thao tác xong gọi `_loadTasks()` để lấy dữ liệu thật từ DB → chắc chắn đúng.

Bước 8: Nối `main.dart`

Tạo/ghi file `lib/main.dart`

dart

```dart
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const SmartTaskManagerApp());
}

class SmartTaskManagerApp extends StatelessWidget {
  const SmartTaskManagerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Smart Task Manager SQLite',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
  }
}
```

7️⃣ 🧪 CÁCH KIỂM CHỨNG (bắt buộc làm)

Test 1: CRUD bình thường

1. Nhập "Học sqflite" → bấm Thêm
2. Tick hoàn thành
3. Bấm vào item để sửa title
4. Xoá 1 item

✅ Nếu UI cập nhật đúng và không crash → CRUD OK.

Test 2: Persistence (quan trọng nhất)

1. Thêm 3 tasks
2. Tick done 1 task
3. Tắt app hoàn toàn (swipe away / stop app)
4. Mở lại

✅ Nếu danh sách và trạng thái còn → SQLite lưu thành công.

PhucPT10@fpt.edu.vn

8️⃣ 🔍 CÁCH KIỂM CHỨNG NÂNG CAO (tuỳ chọn)

Cách 1: In log kiểm tra (dễ nhất cho sinh viên)
Bạn có thể tạm thêm `print()` trong `_addTask`, `_toggleDone`, `_deleteTask` để xem flow.

Cách 2: Xem file database (giảng viên/mentor dùng)
- Android: dùng Device File Explorer (Android Studio) tìm trong thư mục app databases.
- Hoặc dùng công cụ như "DB Browser for SQLite" (copy file db ra ngoài).

> Sinh viên mới học có thể bỏ qua phần này nếu thấy khó.

9️⃣ ⚠️ LỖI PHỔ BIẾN (và cách tránh)

1. Quên await khi insert/update/query
   - Triệu chứng: load ra danh sách chưa kịp cập nhật
   - Fix: mọi thao tác DB đều `await`

2. Lưu bool sai kiểu
   - Sai: lưu `true/false` trực tiếp
   - Đúng: `is_done: 1/0`

3. Không set PRIMARY KEY
   - Triệu chứng: trùng dữ liệu, update sai dòng
   - Fix: `id TEXT PRIMARY KEY`

4. Viết sai tên cột
   - Ví dụ model dùng `isDone` nhưng DB là `is_done`
   - Fix: thống nhất cột DB, dùng đúng key trong toMap/fromMap

5. Gọi load trong build()
   - Fix: gọi trong `initState()` hoặc sau khi thao tác CRUD

🔟 🎓 BÀI HỌC CHÍNH (so sánh SharedPreferences vs SQLite)

- SharedPreferences:
  - ✅ dễ dùng
  - ✅ phù hợp dữ liệu nhỏ
  - ❌ không truy vấn tốt, dữ liệu lớn sẽ bất tiện

- SQLite:
  - ✅ lưu dữ liệu lớn, có cấu trúc bảng
  - ✅ query, sort, filter tốt hơn
  - ❌ code nhiều hơn, cần hiểu bảng/cột/kiểu dữ liệu

PhucPT10@fpt.edu.vn

1️⃣1️⃣ ✅ TỔNG HỢP FULL CODE (tất cả file)

1) lib/main.dart
(đã đưa ở Bước 8)

2) lib/models/task.dart
(đã đưa ở Bước 4)

3) lib/db/app_database.dart
(đã đưa ở Bước 5)

4) lib/repositories/task_repository.dart
(đã đưa ở Bước 6)

5) lib/screens/home_screen.dart
(đã đưa ở Bước 7)

1️⃣2️⃣ 📦 CÁC PHIÊN BẢN CẬP NHẬT THEO TỪNG BƯỚC (theo diễn biến thực hiện)

Version 0 – App chạy được
- Chạy counter mặc định

Version 1 – Tạo model Task + UI List (RAM)
- Mới chỉ lưu trong biến `tasks` (mất khi tắt app)

Version 2 – Tạo DB + tạo table tasks
- Có `AppDatabase`, chạy app không lỗi

Version 3 – Insert + getAll
- Bấm Thêm → insert vào DB
- `_loadTasks()` lấy từ DB ra hiển thị

Version 4 – Update (toggle done + edit title)
- Tick checkbox → update DB
- Bấm item → sửa title → update DB

Version 5 – Delete + DeleteAll
- Xóa 1 task / xóa toàn bộ

✅ Kết thúc Version 5 là bài hoàn chỉnh.

1️⃣3️⃣ 📝 Bài tập thêm (khuyến nghị)

1. Thêm bộ lọc: All / Done / Undone (chỉ UI lọc hoặc query DB)
2. Thêm search theo title
3. Thêm “ưu tiên” (priority: 1-3) và sort theo priority
4. Thêm “deadline” (int milliseconds) và đổi màu nếu gần hạn

PhucPT10@fpt.edu.vn

PHẦN 3 HIVE
SMART TASK MANAGER (Offline-first)

Tài liệu này hướng dẫn sinh viên mới học thực hành lưu dữ liệu offline bằng Hive .
Hive là NoSQL database chạy local (không cần internet), nhanh , dễ dùng, phù hợp cho app Flutter lưu dữ liệu dạng object.

1️⃣ TÓM TẮT THÔNG TIN DỰ ÁN

📌 Tên dự án
Smart Task Manager – Hive Persistence

🎯 Mục tiêu học tập (học xong phải làm được)
1. Hiểu Hive là gì, khác gì so với:
   - SharedPreferences (key–value đơn giản)
   - SQLite (quan hệ bảng/cột, SQL)
2. Biết cách:
   - Cài đặt Hive
   - Khởi tạo Hive trong `main()`
   - Tạo Box
3. Biết 2 cách lưu dữ liệu với Hive:
   - Cách A (dễ, phù hợp mới học): lưu `Task` dưới dạng Map/Json (không cần adapter)
   - Cách B (chuẩn, nâng cao): dùng TypeAdapter để lưu object trực tiếp
4. Thực hành CRUD đầy đủ:
   - Thêm / đọc / sửa / xoá / toggle done
5. Biết kiểm chứng persistence: đóng app mở lại dữ liệu vẫn còn

> Trong tài liệu này sẽ dạy Cách A trước (đơn giản), rồi cuối tài liệu có phần "Nâng cao – TypeAdapter".

2️⃣ 🧩 CHỨC NĂNG
- CRUD task đầy đủ
- Toggle hoàn thành
- Tự load khi mở app
- Lưu ngay sau mỗi thao tác
- (Tuỳ chọn) Xóa tất cả

3️⃣ 🛠 Công nghệ sử dụng
- Flutter SDK
- VS Code
- Packages:
  - `hive`
  - `hive_flutter` (hỗ trợ init + path dễ hơn)
  - (Tuỳ chọn) `build_runner`, `hive_generator` cho TypeAdapter

PhucPT10@fpt.edu.vn

4️⃣ 📂 CẤU TRÚC THƯ MỤC (đề xuất)

text
```
lib/
  ├─── main.dart
  ├─── models/
  │     └─── task.dart
  ├─── services/
  │     └─── hive_service.dart
  └─── screens/
        └─── home_screen.dart
```

Giải thích:
- `hive_service.dart`: mở box + CRUD
- `task.dart`: model + toMap/fromMap (vì ta lưu Map)

5️⃣ 📋 TÓM TẮT DANH SÁCH BƯỚC THỰC HIỆN

1. Tạo project Flutter
2. Thêm dependency Hive
3. Tạo model Task (toMap/fromMap)
4. Init Hive trong `main()` và mở Box
5. Viết `HiveService` (CRUD)
6. Tạo UI (giống phần 1, 2 để quen)
7. Test persistence
8. (Nâng cao) Dùng TypeAdapter để lưu object trực tiếp

6️⃣ **CHI TIẾT TỪNG BƯỚC (dẫn dắt từng thao tác)**

Bước 1: Tạo project

bash
```bash
flutter create smart_task_manager_hive
cd smart_task_manager_hive
code .
```

Chạy thử:

bash
```bash
flutter run
```

✅ Nếu app counter chạy → OK.

Bước 2: Thêm dependency

Mở `pubspec.yaml`:

PhucPT10@fpt.edu.vn

yaml

```yaml
dependencies:
  flutter:
    sdk: flutter
  hive: ^2.2.3
  hive_flutter: ^1.1.0
```

Chạy:

bash

```bash
flutter pub get
```

✅ **Kiểm chứng:**

- Không lỗi pub get
- Import được `package:hive_flutter/hive_flutter.dart`

Bước 3: Tạo Model Task (lưu dưới dạng Map)

Tạo file `lib/models/task.dart`

dart

```dart
class Task {
  final String id;
  String title;
  bool isDone;
  final int createdAt;

  Task({
    required this.id,
    required this.title,
    this.isDone = false,
    required this.createdAt,
  });

  Map<String, dynamic> toMap() => {
        'id': id,
        'title': title,
        'isDone': isDone,
        'createdAt': createdAt,
      };

  factory Task.fromMap(Map map) => Task(
        id: map['id'] as String,
        title: map['title'] as String,
        isDone: (map['isDone'] as bool?) ?? false,
        createdAt: (map['createdAt'] as int?) ?? 0,
      );
}
```


PhucPT10@fpt.edu.vn

✅ **Vì sao dùng Map?**
- Cách này dễ nhất cho sinh viên mới: không cần code generator.
- Hive lưu được nhiều kiểu, trong đó có `Map` và `List`.

Bước 4: Init Hive trong `main()` và mở Box

4.1 Khái niệm Box (giải thích dễ hiểu)
- Box giống như "một cái tủ" lưu dữ liệu.
- Mỗi item trong box là một "ngăn".
- Ta có thể lưu theo key (string/int) hoặc theo index.

Ở đây ta tạo box tên: `taskBox`

Tạo file `lib/main.dart`:

dart
```
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';
import 'screens/home_screen.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Bắt buộc: init hive trước khi mở box
  await Hive.initFlutter();

  // Mở box (giống như "mở tủ dữ liệu")
  await Hive.openBox('taskBox');

  runApp(const SmartTaskManagerApp());
}

class SmartTaskManagerApp extends StatelessWidget {
  const SmartTaskManagerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Smart Task Manager Hive',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.orange),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
  }
}
```


PhucPT10@fpt.edu.vn

✅ **Kiểm chứng:**
- Nếu quên `WidgetsFlutterBinding.ensureInitialized()` có thể bị lỗi khi init.
- Nếu quên `Hive.openBox()` thì CRUD sẽ lỗi vì box chưa mở.

Bước 5: Tạo HiveService (CRUD)

Tạo file `lib/services/hive_service.dart`

dart
```dart
import 'package:hive_flutter/hive_flutter.dart';
import '../models/task.dart';

class HiveService {
  static const String boxName = 'taskBox';
  static const String keyTaskList = 'task_list';

  Box get _box => Hive.box(boxName);

  /// Load danh sách Task từ Hive
  List<Task> loadTasks() {
    final raw = _box.get(keyTaskList);

    if (raw == null) return [];

    // raw kỳ vọng là List<Map>
    final List list = raw as List;
    return list.map((e) => Task.fromMap(e as Map)).toList();
  }

  /// Save danh sách Task xuống Hive
  Future<void> saveTasks(List<Task> tasks) async {
    final listMap = tasks.map((t) => t.toMap()).toList();
    await _box.put(keyTaskList, listMap);
  }

  Future<void> clearTasks() async {
    await _box.delete(keyTaskList);
  }
}
```

✅ **Giải thích cho sinh viên:**
- Với cách A, ta vẫn lưu "cả danh sách" giống SharedPreferences, nhưng:
  - Hive lưu nhanh hơn
  - Không cần encode JSON string
- `put(key, value)` → lưu 1 giá trị theo key.
- `get(key)` → đọc lại.

> Lưu ý: Đây là cách đơn giản , phù hợp mới học. Khi dữ liệu rất lớn, ta sẽ dùng "mỗi task là 1 record" (phần nâng cao).

PhucPT10@fpt.edu.vn

Bước 6: Tạo UI (HomeScreen)

Tạo file `lib/screens/home_screen.dart`

dart

```
import 'package:flutter/material.dart';
import '../models/task.dart';
import '../services/hive_service.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final HiveService service = HiveService();
  final TextEditingController addController = TextEditingController();

  List<Task> tasks = [];
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    _loadData();
  }

  @override
  void dispose() {
    addController.dispose();
    super.dispose();
  }

  void _loadData() {
    // loadTasks() là sync (đọc local nhanh)
    final loaded = service.loadTasks();
    setState(() {
      tasks = loaded;
      isLoading = false;
    });
  }

  Future<void> _saveData() async {
    await service.saveTasks(tasks);
  }

  Future<void> _addTask() async {
    final title = addController.text.trim();
    if (title.isEmpty) return;
```


PhucPT10@fpt.edu.vn

```
    final task = Task(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      title: title,
      createdAt: DateTime.now().millisecondsSinceEpoch,
    );

    setState(() {
      tasks.add(task);
      addController.clear();
    });

    await _saveData();
  }

  Future<void> _toggleDone(Task task) async {
    setState(() {
      task.isDone = !task.isDone;
    });
    await _saveData();
  }

  Future<void> _deleteTask(Task task) async {
    setState(() {
      tasks.removeWhere((t) => t.id == task.id);
    });
    await _saveData();
  }

  Future<void> _editTaskDialog(Task task) async {
    final controller = TextEditingController(text: task.title);

    final result = await showDialog<String>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Sửa task'),
        content: TextField(
          controller: controller,
          autofocus: true,
          decoration: const InputDecoration(
            hintText: 'Nhập tiêu đề mới',
          ),
          onSubmitted: (value) => Navigator.pop(context, value),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, null),
            child: const Text('Hủy'),
          ),
          ElevatedButton(
            onPressed: () => Navigator.pop(context, controller.text),
            child: const Text('Lưu'),
          ),
        ],
```


PhucPT10@fpt.edu.vn

```
    ),
  );

  if (result == null) return;
  final newTitle = result.trim();
  if (newTitle.isEmpty) return;

  setState(() {
    task.title = newTitle;
  });
  await _saveData();
}

Future<void> _clearAllDialog() async {
  final ok = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Xóa tất cả?'),
      content: const Text('Bạn có chắc muốn xóa toàn bộ task không?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Không'),
        ),
        ElevatedButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Có'),
        ),
      ],
    ),
  );

  if (ok != true) return;

  setState(() {
    tasks.clear();
  });
  await service.clearTasks();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Smart Task Manager (Hive)'),
      actions: [
        IconButton(
          onPressed: _clearAllDialog,
          icon: const Icon(Icons.delete_forever),
          tooltip: 'Xóa tất cả',
        ),
      ],
    ),
```


PhucPT10@fpt.edu.vn

```
      body: isLoading
          ? const Center(child: CircularProgressIndicator())
          : Column(
              children: [
                Padding(
                  padding: const EdgeInsets.all(12),
                  child: Row(
                    children: [
                      Expanded(
                        child: TextField(
                          controller: addController,
                          decoration: const InputDecoration(
                            labelText: 'Nhập task mới',
                            border: OutlineInputBorder(),
                          ),
                          onSubmitted: (_) => _addTask(),
                        ),
                      ),
                      const SizedBox(width: 10),
                      ElevatedButton(
                        onPressed: _addTask,
                        child: const Text('Thêm'),
                      )
                    ],
                  ),
                ),
                const Divider(height: 0),
                Expanded(
                  child: tasks.isEmpty
                      ? const Center(child: Text('Chưa có task nào.'))
                      : ListView.separated(
                          itemCount: tasks.length,
                          separatorBuilder: (_, __) => const Divider(height:
0),
                          itemBuilder: (context, index) {
                            final task = tasks[index];
                            return ListTile(
                              leading: Checkbox(
                                value: task.isDone,
                                onChanged: (_) => _toggleDone(task),
                              ),
                              title: Text(
                                task.title,
                                style: TextStyle(
                                  decoration: task.isDone
                                      ? TextDecoration.lineThrough
                                      : null,
                                ),
                              ),
                              onTap: () => _editTaskDialog(task),
                              trailing: IconButton(
                                icon: const Icon(Icons.delete),
                                onPressed: () => _deleteTask(task),
```


PhucPT10@fpt.edu.vn

```
                                ),
                              );
                            },
                          ),
                      ),
                    ],
                  ),
      );
    }
}
```

✅ Vì sao `_loadData()` không async?
- Vì `HiveService.loadTasks()` đọc dữ liệu local nhanh (đồng bộ).
- Nhưng `saveTasks()` vẫn async vì ghi file.

7️⃣ 🧪 CÁCH KIỂM CHỨNG (bắt buộc)

Test 1: CRUD
1. Thêm 2–3 task
2. Tick done 1 task
3. Sửa title 1 task
4. Xoá 1 task

✅ Nếu UI cập nhật đúng → CRUD OK.

Test 2: Persistence
1. Thêm task
2. Tắt app hoàn toàn
3. Mở lại app

✅ Nếu task còn → Hive lưu OK.

8️⃣ ⚠️ LỖI PHỔ BIẾN (và cách tránh)

1. Quên init Hive
   - Lỗi: Hive not initialized
   - Fix: `await Hive.initFlutter();`

2. Quên openBox
   - Lỗi: Box not found
   - Fix: `await Hive.openBox('taskBox');`

3. Lưu kiểu dữ liệu sai
   - Ví dụ lưu `List<Task>` trực tiếp (khi chưa dùng adapter)
   - Fix: lưu `List<Map>` hoặc dùng TypeAdapter

PhucPT10@fpt.edu.vn

4. Ép kiểu (cast) sai khi load
   - Fix: kiểm tra `raw == null`, rồi cast từng phần tử về `Map`

9️⃣ 🎓 BÀI HỌC RÚT RA

- Hive mạnh hơn SharedPreferences vì:
  - Không cần JSON encode/decode (khi lưu kiểu cơ bản/Map)
  - Nhanh và tiện cho local NoSQL
- Khi dữ liệu phức tạp/lớn, nên:
  - dùng TypeAdapter
  - hoặc lưu mỗi task như 1 record riêng (không lưu cả list)

🔟 ✅ TỔNG HỢP FULL CODE (tất cả file)

1) lib/main.dart

dart

```
import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';
import 'screens/home_screen.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  await Hive.openBox('taskBox');

  runApp(const SmartTaskManagerApp());
}

class SmartTaskManagerApp extends StatelessWidget {
  const SmartTaskManagerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Smart Task Manager Hive',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.orange),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
  }
}


   2) lib/models/task.dart
 dart
```


PhucPT10@fpt.edu.vn

```
class Task {
  final String id;
  String title;
  bool isDone;
  final int createdAt;

  Task({
    required this.id,
    required this.title,
    this.isDone = false,
    required this.createdAt,
  });

  Map<String, dynamic> toMap() => {
        'id': id,
        'title': title,
        'isDone': isDone,
        'createdAt': createdAt,
      };

  factory Task.fromMap(Map map) => Task(
        id: map['id'] as String,
        title: map['title'] as String,
        isDone: (map['isDone'] as bool?) ?? false,
        createdAt: (map['createdAt'] as int?) ?? 0,
      );
}
```

3) lib/services/hive_service.dart

dart

```
import 'package:hive_flutter/hive_flutter.dart';
import '../models/task.dart';

class HiveService {
  static const String boxName = 'taskBox';
  static const String keyTaskList = 'task_list';

  Box get _box => Hive.box(boxName);

  List<Task> loadTasks() {
    final raw = _box.get(keyTaskList);
    if (raw == null) return [];

    final List list = raw as List;
    return list.map((e) => Task.fromMap(e as Map)).toList();
  }

  Future<void> saveTasks(List<Task> tasks) async {
    final listMap = tasks.map((t) => t.toMap()).toList();
    await _box.put(keyTaskList, listMap);
  }
```

```
  Future<void> clearTasks() async {
    await _box.delete(keyTaskList);
  }
}
```

4) lib/screens/home_screen.dart
(Chính là file ở Bước 6)

1️⃣1️⃣ 📦 CÁC PHIÊN BẢN CẬP NHẬT THEO TỪNG BƯỚC

Version 0 – App chạy được
- run counter

Version 1 – Tạo model + UI danh sách (RAM)
- tasks lưu trong biến

Version 2 – Init Hive + openBox
- app chạy không lỗi Hive

Version 3 – Save/Load danh sách task bằng Hive (List<Map>)
- đóng mở app vẫn còn dữ liệu

Version 4 – Hoàn chỉnh CRUD + clear all
- thêm/sửa/xóa/toggle done + xóa tất cả

1️⃣2️⃣ 🚀 Nâng cao (tuỳ chọn): Lưu object trực tiếp bằng TypeAdapter

> Mục tiêu: không phải lưu `Map` nữa, mà lưu `Task` trực tiếp vào Hive.

Cài thêm dependency (nâng cao)
yaml
```yaml
dev_dependencies:
  build_runner: ^2.4.10
  hive_generator: ^2.0.1
```

Ý tưởng
- Thêm annotation `@HiveType(typeId: 0)` và `@HiveField(n)`
- Chạy generate:
bash
```bash
flutter pub run build_runner build
```


PhucPT10@fpt.edu.vn

PHẦN 4 – DRIFT (SQLite ORM)
SMART TASK MANAGER (Offline-first)

Tài liệu này hướng dẫn sinh viên mới học thực hành lưu dữ liệu offline bằng Drift (một ORM/Query Builder cho SQLite trong Flutter).

Drift giúp bạn viết code Dart để tạo bảng và truy vấn thay vì viết SQL thủ công như sqflite, đồng thời hỗ trợ type-safe (ít lỗi hơn).

1️⃣ TÓM TẮT THÔNG TIN DỰ ÁN

📌 Tên dự án

Smart Task Manager – Drift Persistence

🎯 Mục tiêu học tập (học xong phải làm được)

1. Hiểu Drift là gì và dùng để làm gì:
   - Drift vẫn lưu bằng SQLite , nhưng thao tác theo kiểu "Dart code + type-safe".
2. Biết cài đặt drift + build_runner và generate code.
3. Biết tạo:
   - `Table` (bảng tasks)
   - `AppDatabase`
   - `DAO/Repository` (CRUD)
4. Thực hành CRUD đầy đủ:
   - thêm / xem / sửa / xoá / toggle done
5. Hiểu cơ bản về:
   - migration (version DB)
   - watch stream (tự cập nhật UI) *(tuỳ chọn)*

2️⃣ 🧩 CHỨC NĂNG

- CRUD task đầy đủ
- Toggle hoàn thành
- Dữ liệu còn khi tắt/mở app
- (Tuỳ chọn) Tự cập nhật UI bằng Stream (watch)

3️⃣ 🛠 Công nghệ sử dụng

- Flutter SDK
- VS Code
- Packages:
  - `drift`
  - `drift_flutter`
  - `sqlite3_flutter_libs` (giúp SQLite chạy ổn định trên mobile/desktop)
  - Dev:
    - `drift_dev`
    - `build_runner`

> Lưu ý: drift cần generate code, vì vậy phải chạy build_runner.

PhucPT10@fpt.edu.vn

4️⃣ 📂 CẤU TRÚC THƯ MỤC (đề xuất dễ học)

text

```
lib/
  ├─── main.dart
  ├─── db/
  │     ├─── app_database.dart
  │     └─── tables.dart
  ├─── repositories/
  │     └─── task_dao.dart
  └─── screens/
        └─── home_screen.dart
```

Giải thích:

- `tables.dart`: khai báo bảng `Tasks`
- `app_database.dart`: database chính (khởi tạo drift)
- `task_dao.dart`: nơi viết CRUD (giống repository/DAO)
- `home_screen.dart`: UI gọi dao

5️⃣ 📋 TÓM TẮT DANH SÁCH BƯỚC THỰC HIỆN

1. Tạo project
2. Thêm dependency drift + dev tools
3. Tạo bảng `Tasks` bằng Dart (không viết SQL)
4. Tạo `AppDatabase` và generate code
5. Viết `TaskDao` CRUD
6. Tạo UI (giống các phần trước)
7. Test persistence
8. (Tuỳ chọn) `watchTasks()` để UI tự cập nhật

6️⃣ CHI TIẾT TỪNG BƯỚC (dẫn dắt từng thao tác)

Bước 1: Tạo project

bash

```bash
flutter create smart_task_manager_drift
cd smart_task_manager_drift
code .
```

Chạy thử:

bash

```bash
flutter run
```

Bước 2: Thêm dependency drift

Mở `pubspec.yaml` thêm:

PhucPT10@fpt.edu.vn

yaml

```yaml
dependencies:
  flutter:
    sdk: flutter

  drift: ^2.20.0
  drift_flutter: ^2.20.0
  sqlite3_flutter_libs: ^0.5.26
  path: ^1.9.0
  path_provider: ^2.1.4

dev_dependencies:
  flutter_test:
    sdk: flutter

  drift_dev: ^2.20.0
  build_runner: ^2.4.12
```

Chạy:

bash

```bash
flutter pub get
```

✅ **Kiểm chứng:**

- pub get không lỗi.

Bước 3: Tạo bảng Tasks (tables.dart)

Tạo file `lib/db/tables.dart`

dart

```dart
import 'package:drift/drift.dart';

class Tasks extends Table {
  TextColumn get id => text()(); // primary key dạng text
  TextColumn get title => text()();

  /// bool trong drift có thể dùng boolean()
  BoolColumn get isDone => boolean().withDefault(const Constant(false))();

  /// thời gian tạo (millisecondsSinceEpoch)
  IntColumn get createdAt => integer()();

  @override
  Set<Column> get primaryKey => {id};
}
```

✅ **Giải thích cho sinh viên:**

- Drift cho phép khai báo cột bằng Dart:
  - `text()`, `integer()`, `boolean()`
- `primaryKey` giống như SQLite PRIMARY KEY.

Bước 4: Tạo AppDatabase + generate code

Tạo file `lib/db/app_database.dart`

dart
```dart
import 'dart:io';
import 'package:drift/drift.dart';
import 'package:drift_flutter/drift_flutter.dart';
import 'package:path_provider/path_provider.dart';
import 'package:path/path.dart' as p;

import 'tables.dart';

part 'app_database.g.dart';

@DriftDatabase(tables: [Tasks])
class AppDatabase extends _$AppDatabase {
  AppDatabase() : super(_openConnection());

  @override
  int get schemaVersion => 1;
}

LazyDatabase _openConnection() {
  return LazyDatabase(() async {
    final dir = await getApplicationDocumentsDirectory();
    final file = File(p.join(dir.path, 'smart_task_manager_drift.sqlite'));
    return NativeDatabase.createInBackground(file);
  });
}
```

4.1 Chạy generate code

Mở Terminal:

bash
```bash
flutter pub run build_runner build --delete-conflicting-outputs
```

✅ Kiểm chứng:
- Trong `lib/db/` sẽ sinh ra file `app_database.g.dart`
- Nếu lỗi, thường do:
  - thiếu `part 'app_database.g.dart';`
  - sai tên file
  - chưa thêm drift_dev/build_runner

Bước 5: Viết DAO/Repository (CRUD)

Tạo file `lib/repositories/task_dao.dart`

dart
```dart
import 'package:drift/drift.dart';
import '../db/app_database.dart';

class TaskDao {
```


PhucPT10@fpt.edu.vn

```
  final AppDatabase db;
  TaskDao(this.db);

  // CREATE
  Future<void> insertTask({
    required String id,
    required String title,
    required int createdAt,
  }) async {
    await db.into(db.tasks).insert(
          TasksCompanion.insert(
            id: id,
            title: title,
            createdAt: createdAt,
          ),
          mode: InsertMode.insertOrReplace,
        );
  }

  // READ (all)
  Future<List<Task>> getAllTasks() {
    return (db.select(db.tasks)
          ..orderBy([(t) => OrderingTerm.desc(t.createdAt)]))
        .get();
  }

  // UPDATE (toggle done)
  Future<void> toggleDone(Task task) async {
    await (db.update(db.tasks)..where((t) => t.id.equals(task.id))).write(
      TasksCompanion(
        isDone: Value(!task.isDone),
      ),
    );
  }

  // UPDATE (edit title)
  Future<void> updateTitle({
    required String id,
    required String newTitle,
  }) async {
    await (db.update(db.tasks)..where((t) => t.id.equals(id))).write(
      TasksCompanion(
        title: Value(newTitle),
      ),
    );
  }

  // DELETE
  Future<void> deleteTask(String id) async {
    await (db.delete(db.tasks)..where((t) => t.id.equals(id))).go();
  }

  Future<void> deleteAll() async {
    await db.delete(db.tasks).go();
  }

  // (Tuỳ chọn) WATCH: UI tự cập nhật khi DB đổi
  Stream<List<Task>> watchAllTasks() {
    return (db.select(db.tasks)
          ..orderBy([(t) => OrderingTerm.desc(t.createdAt)]))
        .watch();
  }
}
```

✅ **Giải thích:**

- `Task` ở đây là data class do drift generate (không cần tự tạo model).
- CRUD thao tác qua:
  - `into(db.tasks).insert(...)`
  - `select(db.tasks).get()`
  - `update(db.tasks).write(...)`
  - `delete(db.tasks).go()`

Bước 6: Tạo UI (HomeScreen)

Cách học dễ nhất: load thủ công (không dùng Stream)

Tạo file `lib/screens/home_screen.dart`

dart

```
import 'package:flutter/material.dart';
import '../db/app_database.dart';
import '../repositories/task_dao.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  late final AppDatabase db;
  late final TaskDao dao;

  final TextEditingController addController = TextEditingController();

  List<Task> tasks = [];
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    db = AppDatabase();
    dao = TaskDao(db);
    _loadTasks();
  }

  @override
  void dispose() {
    addController.dispose();
    db.close();
    super.dispose();
  }

  Future<void> _loadTasks() async {
    final loaded = await dao.getAllTasks();
    setState(() {
      tasks = loaded;
      isLoading = false;
    });
  }
```

```
Future<void> _addTask() async {
  final title = addController.text.trim();
  if (title.isEmpty) return;

  final now = DateTime.now().millisecondsSinceEpoch;

  await dao.insertTask(
    id: now.toString(),
    title: title,
    createdAt: now,
  );

  addController.clear();
  await _loadTasks();
}

Future<void> _toggleDone(Task task) async {
  await dao.toggleDone(task);
  await _loadTasks();
}

Future<void> _deleteTask(Task task) async {
  await dao.deleteTask(task.id);
  await _loadTasks();
}

Future<void> _editTaskDialog(Task task) async {
  final controller = TextEditingController(text: task.title);

  final result = await showDialog<String>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Sửa task'),
      content: TextField(
        controller: controller,
        autofocus: true,
        decoration: const InputDecoration(hintText: 'Nhập tiêu đề mới'),
        onSubmitted: (value) => Navigator.pop(context, value),
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, null),
          child: const Text('Hủy'),
        ),
        ElevatedButton(
          onPressed: () => Navigator.pop(context, controller.text),
          child: const Text('Lưu'),
        ),
      ],
    ),
  );

  if (result == null) return;
  final newTitle = result.trim();
  if (newTitle.isEmpty) return;

  await dao.updateTitle(id: task.id, newTitle: newTitle);
  await _loadTasks();
}

Future<void> _clearAllDialog() async {
  final ok = await showDialog<bool>(
```


PhucPT10@fpt.edu.vn

```
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Xóa tất cả?'),
      content: const Text('Bạn có chắc muốn xóa toàn bộ task không?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Không'),
        ),
        ElevatedButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Có'),
        ),
      ],
    ),
  );

  if (ok != true) return;

  await dao.deleteAll();
  await _loadTasks();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Smart Task Manager (Drift)'),
      actions: [
        IconButton(
          onPressed: _clearAllDialog,
          icon: const Icon(Icons.delete_forever),
        )
      ],
    ),
    body: isLoading
        ? const Center(child: CircularProgressIndicator())
        : Column(
            children: [
              Padding(
                padding: const EdgeInsets.all(12),
                child: Row(
                  children: [
                    Expanded(
                      child: TextField(
                        controller: addController,
                        decoration: const InputDecoration(
                          labelText: 'Nhập task mới',
                          border: OutlineInputBorder(),
                        ),
                        onSubmitted: (_) => _addTask(),
                      ),
                    ),
                    const SizedBox(width: 10),
                    ElevatedButton(
                      onPressed: _addTask,
                      child: const Text('Thêm'),
                    ),
                  ],
                ),
              ),
              const Divider(height: 0),
              Expanded(
```


PhucPT10@fpt.edu.vn

```
                  child: tasks.isEmpty
                      ? const Center(child: Text('Chưa có task nào.'))
                      : ListView.separated(
                          itemCount: tasks.length,
                          separatorBuilder: (_, __) => const Divider(height: 0),
                          itemBuilder: (context, index) {
                            final task = tasks[index];
                            return ListTile(
                              leading: Checkbox(
                                value: task.isDone,
                                onChanged: (_) => _toggleDone(task),
                              ),
                              title: Text(
                                task.title,
                                style: TextStyle(
                                  decoration: task.isDone
                                      ? TextDecoration.lineThrough
                                      : null,
                                ),
                              ),
                              onTap: () => _editTaskDialog(task),
                              trailing: IconButton(
                                icon: const Icon(Icons.delete),
                                onPressed: () => _deleteTask(task),
                              ),
                            );
                          },
                        ),
                  ),
                ],
              ),
    );
  }
}
```

Bước 7: Nối `main.dart`

Tạo file `lib/main.dart`

dart

```dart
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const SmartTaskManagerApp());
}

class SmartTaskManagerApp extends StatelessWidget {
  const SmartTaskManagerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Smart Task Manager Drift',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
```

```
    );
  }
}
```

✅ **Kiểm chứng:**
- Chạy app không lỗi
- Thêm task hiển thị được

7️⃣ 🧪 CÁCH KIỂM CHỨNG (bắt buộc)

Test 1: CRUD
1. Thêm 3 task
2. Tick done 1 task
3. Sửa title 1 task
4. Xóa 1 task

✅ Nếu mọi thao tác đúng → CRUD OK.

Test 2: Persistence
1. Tắt app hoàn toàn
2. Mở lại app

✅ Nếu task còn → drift lưu bằng SQLite OK.

8️⃣ ⚠️ LỖI PHỔ BIẾN (và cách tránh)

1. Chưa chạy build_runner
   - Lỗi: thiếu `_$AppDatabase`, thiếu class `Task`
   - Fix:
     bash
     flutter pub run build_runner build --delete-conflicting-outputs

2. Quên `part 'app_database.g.dart';`
   - Lỗi generate không đúng
   - Fix: kiểm tra lại file `app_database.dart`

3. Tạo bảng nhưng quên primary key
   - Drift cần primary key để update/delete theo id cho dễ

4. Không đóng database
   - Fix: trong `dispose()` gọi `db.close()`

PhucPT10@fpt.edu.vn

9️⃣ 🎓 BÀI HỌC RÚT RA (so sánh nhanh)

- sqflite: bạn tự viết SQL + tự map dữ liệu
- drift:
  - ✅ type-safe
  - ✅ generate data class tự động
  - ✅ query viết bằng Dart, ít sai tên cột
  - ❌ cần code generation (build_runner)

🔟 ✅ TỔNG HỢP FULL CODE (tất cả file)

1) lib/main.dart
(đã đưa ở Bước 7)

2) lib/db/tables.dart
(đã đưa ở Bước 3)

3) lib/db/app_database.dart
(đã đưa ở Bước 4)

4) lib/repositories/task_dao.dart
(đã đưa ở Bước 5)

5) lib/screens/home_screen.dart
(đã đưa ở Bước 6)

> Lưu ý: drift còn sinh file `lib/db/app_database.g.dart` (không cần copy vào tài liệu vì nó do tool tạo).

1️⃣1️⃣ 🧱 CÁC PHIÊN BẢN CẬP NHẬT THEO TỪNG BƯỚC

Version 0 – App chạy được
- chạy counter

Version 1 – Tạo Tables + AppDatabase (chưa CRUD)
- chạy build_runner ra `g.dart`

Version 2 – Viết DAO: insert + getAll
- thêm task và load hiển thị

Version 3 – Update + Delete
- toggle done, edit title, delete

Version 4 – Hoàn thiện UI + deleteAll
- full CRUD hoàn chỉnh

PhucPT10@fpt.edu.vn

Version 5 (tuỳ chọn) – Dùng watch() để UI tự cập nhật
- thay `_loadTasks()` bằng StreamBuilder + `dao.watchAllTasks()`

1️⃣ 2️⃣ Bài tập thêm (khuyến nghị)
1. Thêm filter All/Done/Undone bằng query drift
2. Thêm search theo title (`title.like('%abc%')`)
3. Thêm cột priority (int) + migration schemaVersion=2
4. Thêm trang thống kê: tổng task, done, undone

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

Thuật ngữ Định nghĩa dễ hiểu Dùng trong Flutter khi nào? Hiểu sai phổ biến

NHÓM 1: DATA & STORAGE FOUNDATION

Thuật ngữ	Định nghĩa	Ứng dụng	Hiểu sai
Local Persistence	Lưu dữ liệu trên thiết bị để không mất khi đóng app	Lưu token, danh sách offline	Nghĩ rằng lưu RAM là persistence
In-memory State	Dữ liệu chỉ tồn tại trong RAM	Biến trong Provider	Nhầm với lưu vĩnh viễn
Key–Value Storage	Lưu theo cặp key → value	shared_preferences	Dùng cho danh sách lớn
Relational Database	CSDL có bảng và quan hệ	sqflite	Nghĩ là phức tạp không cần thiết
NoSQL Database	CSDL không quan hệ bảng	hive	Nghĩ NoSQL luôn nhanh hơn SQL
ORM (Object Relational Mapping)	Công cụ ánh xạ table ↔ object	drift	Nghĩ ORM là database mới
Migration	Cập nhật cấu trúc DB khi nâng version	Khi thêm cột	Bỏ qua → crash production
Serialization	Chuyển object → JSON/String	Export/Backup	Nghĩ JSON = database
Deserialization	JSON → object	Load cache	Không validate dữ liệu

NHÓM 2: NETWORK & RETRY

Thuật ngữ	Định nghĩa	Ứng dụng	Hiểu sai
Retry Logic	Thử lại request khi thất bại	API timeout	Retry vô hạn
Exponential Backoff	Delay tăng dần sau mỗi lần retry	Production API	Retry liên tục không delay
Timeout	Thời gian tối đa chờ phản hồi	http request	Không cấu hình timeout
Idempotent Request	Gọi lại nhiều lần vẫn không gây lỗi	GET API	Retry POST thanh toán
Network-first	Ưu tiên lấy từ server	Trang thanh toán	Dùng cho mọi màn hình
Cache-first	Ưu tiên dữ liệu local	Trang danh sách	Không update lại server
Stale-while-revalidate	Hiển thị cache, đồng thời gọi API update	Feed tin tức	Không hiểu cơ chế 2 luồng

NHÓM 3: REACTIVE & STREAM

Thuật ngữ	Định nghĩa	Ứng dụng	Hiểu sai
Stream	Dòng dữ liệu thay đổi theo thời gian	Reactive UI	Nghĩ Stream = Future
Future	Kết quả bất đồng bộ 1 lần	Gọi API 1 lần	Dùng cho dữ liệu cần update liên tục
Reactive Programming	UI tự cập nhật khi data thay đổi	Drift watch()	Dùng setState mọi nơi

PhucPT10@fpt.edu.vn

StreamBuilder	Widget lắng nghe Stream	Hiển thị DB live	Không xử lý snapshot null
Subscription	Đăng ký lắng nghe stream	listen()	Không dispose
Hot Stream	Stream phát dữ liệu liên tục	WebSocket	Nhầm với Future
Cold Stream	Stream bắt đầu khi có listener	Drift watch()	Không hiểu cơ chế khởi tạo

NHÓM 4: SESSION & TOKEN

Thuật ngữ	Định nghĩa	Ứng dụng	Hiểu sai
Session State	Trạng thái đăng nhập hiện tại	Guarded route	Nghĩ là lưu DB
Access Token	Chuỗi xác thực user	API Authorization	Lưu plain text không bảo mật
Token Persistence	Lưu token local	Auto login	Lưu vào SQLite
Secure Storage	Lưu dữ liệu mã hóa	Token production	Dùng SharedPreferences cho password
Authentication	Xác thực user	Login	Nhầm với Authorization
Authorization	Phân quyền truy cập	Role-based access	Không phân biệt với auth

NHÓM 5: KIẾN TRÚC & DATA LAYER

Thuật ngữ	Định nghĩa	Ứng dụng	Hiểu sai
Data Layer	Tầng xử lý dữ liệu	Repository pattern	UI gọi DB trực tiếp
Repository Pattern	Lớp trung gian giữa UI & DB/API	Clean architecture	Nghĩ là chỉ để "cho đẹp"
Separation of Concerns	Tách rõ trách nhiệm	UI không gọi SQLite	Viết tất cả trong 1 file
Single Source of Truth	Một nguồn dữ liệu chính	Drift DB làm nguồn chính	Vừa cache vừa state trùng lặp
Offline-first	Thiết kế ưu tiên hoạt động offline	App vùng mạng yếu	Chỉ là thêm cache
Concurrency	Xử lý nhiều tác vụ song song	Async DB	Không hiểu race condition
Race Condition	Lỗi do async chạy không đúng thứ tự	Không await	Nghĩ là lỗi random

NHÓM 6: LỖI PHỔ BIẾN

Thuật ngữ	Định nghĩa	Tình huống	Hiểu sai
Memory Leak	Không giải phóng tài nguyên	Không close stream	Flutter không bao giờ leak
Main Isolate	Luồng chính UI	Query DB lớn	Nghĩ Flutter tự tối ưu
Blocking UI	Làm UI đứng hình	Đọc file lớn	Không hiểu isolate
Crash Production	Lỗi chỉ xuất hiện ngoài đời	Migration sai	Nghĩ test local là đủ
Cold Start	Thời gian mở app	Load cache ban đầu	Không tối ưu load đầu

5 THUẬT NGỮ SINH VIÊN PHẢI NHỚ NHẤT

Nếu chỉ nhớ 5:

Persistence ≠ In-memory
Retry phải có giới hạn
Cache phải có chiến lược
Stream ≠ Future

PhucPT10@fpt.edu.vn

Repository là bắt buộc trong production

📌 Nhận xét thực tế giảng dạy

Sinh viên thường:

Sợ SQL nhưng lạm dụng SharedPreferences

Dùng Future cho dữ liệu cần reactive

Retry mọi lỗi

Không hiểu TTL cache

Sau khi nắm được glossary này → tư duy thay đổi rõ rệt.

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

1 Phân tích chiến lược Retry

So sánh Fixed Delay Retry và Exponential Backoff trong ứng dụng Flutter gọi API production.
Trong trường hợp nào mỗi chiến lược gây hại thay vì có lợi?

2 Đánh giá Caching Strategy

Phân tích sự khác biệt giữa Network-first, Cache-first và Stale-While-Revalidate trong một ứng dụng thương mại điện tử.
Nếu chọn sai chiến lược, UX sẽ bị ảnh hưởng như thế nào?

3 So sánh Hive vs SQLite vs Drift

So sánh hive, sqflite và drift theo các tiêu chí:

- Hiệu năng
- Type safety
- Reactive support
- Migration
- Maintainability

Trong project 1 năm phát triển, công cụ nào phù hợp nhất?

4 Phân tích vấn đề Async & Race Condition

Hãy phân tích một tình huống race condition khi đọc và ghi local DB trong Flutter.
Làm sao để phát hiện và phòng tránh từ thiết kế ban đầu?

5 Reactive vs Non-reactive Data Flow

Phân tích sự khác biệt giữa sử dụng Future và Stream khi làm việc với Drift.
Khi nào Stream gây phức tạp không cần thiết?

6 Single Source of Truth

Giải thích vì sao Drift database nên đóng vai trò Single Source of Truth thay vì lưu state trùng lặp trong Provider.

7 TTL và Data Freshness

Phân tích tác động của việc không có TTL trong cache đối với tính đúng đắn của dữ liệu trong ứng dụng tài chính.

8 Data Layer Architecture

Thiết kế data layer cho ứng dụng Flutter theo hướng Repository Pattern.
Nếu UI gọi SQLite trực tiếp, hệ quả dài hạn là gì?

9 Blocking UI & Isolate

Phân tích vì sao đọc dữ liệu lớn từ SQLite có thể gây block UI.
Khi nào cần chuyển sang isolate riêng?

10 Offline-first Architecture

Phân tích ưu và nhược điểm của kiến trúc offline-first.
Trong bối cảnh mạng 5G phổ biến, offline-first còn cần thiết không?

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

1 Thiết kế Retry trong môi trường API không ổn định
Nếu server downtime 10 phút, hệ thống retry nên hoạt động thế nào để không gây quá tải thêm cho backend?

2 Token Expiration & Auto Refresh
Thiết kế cơ chế lưu và refresh token an toàn trong Flutter.
Làm sao tránh vòng lặp retry vô hạn khi token hết hạn?

3 Cache Versioning
Khi backend thay đổi cấu trúc JSON, cache local cũ sẽ gây lỗi gì?
Thiết kế cơ chế version hóa cache như thế nào?

4 Migration trong ứng dụng có 50.000 người dùng
Nếu thay đổi schema Drift, làm sao đảm bảo migration không làm crash app khi update từ version cũ?

5 Data Corruption
Phân tích các nguyên nhân có thể gây hỏng local database (crash khi đang ghi, mất điện, bug code).
Thiết kế cơ chế recovery như thế nào?

6 Large Dataset Handling
Khi SQLite chứa 100.000 record, pagination nên được thiết kế ra sao để không block UI?

7 Multi-Source Data Sync
Thiết kế kiến trúc khi dữ liệu có thể thay đổi từ:
- Local
- API
- Push notification

Làm sao đảm bảo consistency?

8 Background Sync
Làm sao thực hiện sync dữ liệu nền trong Flutter mà không làm hao pin hoặc crash?

9 Secure Storage vs SharedPreferences
Phân tích rủi ro bảo mật nếu lưu access token trong shared_preferences thay vì secure storage.

10 Crash khi mất mạng
Thiết kế luồng dữ liệu sao cho khi API fail, UI không crash mà vẫn hiển thị dữ liệu cache.

11 Performance Profiling
Làm sao đo lường thời gian đọc SQLite trong Flutter?
Dùng DevTools để phân tích bottleneck như thế nào?

12 Memory Leak do Stream
Khi dùng StreamBuilder và manual subscription song song, những lỗi memory leak nào có thể xảy ra?

13 Scaling App Từ MVP → Production
Khi app tăng từ 1.000 → 100.000 user, những thay đổi nào cần thực hiện ở data layer?

PhucPT10@fpt.edu.vn

1 4 Testing Data Layer

Viết unit test cho repository sử dụng Drift như thế nào để không phụ thuộc database thật?

1 5 Real Production Failure Case Study

Tìm một case thực tế (hoặc tự giả lập) nơi:

- Retry sai
- Cache sai
- Migration lỗi
  - gây crash production.
  - Phân tích nguyên nhân và đề xuất kiến trúc phòng tránh.

PhucPT10@fpt.edu.vn